



图书分类号: TP3

学校代码: 10615

西南石油大学

专业学位硕士学位论文

论 文 题 目 基于区块链的加密信息

备份系统研究与设计

学 位 类 别 工程硕士

专业学位领域 计算机技术

研 究 方 向 区块链

研 究 生 姓 名 庾小忠

学 号 201422000246

校内导师姓名 李建

校外导师姓名

论文完成时间: 2017 年 6 月

摘 要

随着信息技术的快速发展,人们已经进入信息爆炸的时代,人与人之间的合作、竞争和信任都是以信息为载体,信息资源无疑是最宝贵的资源,一旦丢失则损失惨重。传统的信息记载方式无法证明信息所属,信息很容易泄露或丢失,人们生活在一个信息不可完全可信的环境中,使得合法权益受到威胁。区块链作为比特币的技术核心,通过技术手段解决了人与人之间,以及人与系统之间的互信问题,为保护用户信息安全做出了跨时代的尝试。

本文基于区块链技术思想设计了一套加密信息备份系统,并采用 Node.js 及相关组件完成了该系统的开发和测试工作。该加密信息备份系统的核心功能是帮助用户备份重要的文本信息,信息经加密后被存储到所有的应用节点,节点遍布全球,每个节点都拥有最新的完整数据拷贝。同时,本文对该系统的便捷性进行了改良,设计并实现了用户私钥加密备份和文本信息的安全发送等功能。

本文通过设计基于 P2P 网络的分布式数据存储策略,保证了用户备份的文本信息不会丢失;采用基于椭圆曲线系列算法的签名验证技术,实现了信息的所有权认证;采用安全的加解密技术,保证了用户信息的安全;设计了有效的共识机制,保证系统正常运行;设计了可远程访问的应用节点,使得用户随时随地可以操作文本信息。本文通过对区块链技术的应用创新,证明了区块链在去中心化的文本信息管理上的技术可行性和安全可靠。

关键词: 区块链; 加解密技术; P2P; 分布式; Node.js;

Abstract

With the rapid development of information technology, people have entered the era of information explosion, interpersonal cooperation, competition and trust is based on the information as carrier. Information resource is the most valuable resource. Once the information is lost, it will cause heavy losses. The traditional way of recording information cannot prove that the information belongs. As the information is easy to leak or loss. People live in an environment where information cannot be completely trusted, and the legitimate rights and interests are threatened. Block chain as the core technology of bitcoin, which can deal with the mutual trust between people, as well as between people and system, which has made a significant attempt to protect user information security.

Based on the idea of block chain technology, this paper designs a set of encrypted information backup system, and uses Node.js and related components to complete the development and system testing. The core functionality of the encrypted information backup system aims to assist user to backup important text information. Encrypted information will be stored in all application nodes, as well nodes are all around the world. Moreover, each node has a complete copy of the latest data. At the same time, this paper has improved the convenience of the system, designed and implemented the user private key encryption and backup and text information security and other functions.

This paper designs a distributed data storage strategy based on P2P network to ensure that user backup text information is not lost. The signature verification technology based on elliptic curve algorithm is used to realize the information ownership authentication. Using secure encryption and decryption technology aims to ensure the security of the user information. The effective consensus mechanism is designed to ensure the normal operation of the system, and the application nodes can be accessed remotely so that the user can operate the text information anywhere at any time. Based on the application innovation of block chain technology, this paper proves the technical feasibility and safety and reliability of block chain in the process of decentralized text information management.

Key words: Block chain; encryption; P2P ;Distributed storage;Node.js;

目 录

摘 要..... I

Abstract..... 1

目 录..... I

第 1 章 绪论..... 1

1.1 研究背景 1

1.1.1 技术背景1

1.1.2 应用背景1

1.2 国内外研究现状..... 2

1.3 研究的目的及意义..... 3

1.4 研究内容概述..... 4

1.5 论文章节安排..... 4

1.6 本章小结 5

第 2 章 关键技术研究..... 6

2.1 区块链基本概念 6

2.2 密码学技术概述..... 8

2.2.1 公钥私钥体系8

2.2.2 椭圆曲线系列算法.....9

2.3 P2P 网络和分布式存储11

2.3.1 P2P 网络 11

2.3.2 分布式存储 11

2.3.3 二者的关系 12

2.4 共识机制12

2.4.1 POW..... 13

2.4.2 POS..... 13

2.4.3 DPoS..... 13

2.4.4 Pool 13

2.5 Node.js 技术概述.....14

2.6 本章小结14

第3章 系统分析与概要设计15

3.1 系统设计目标15

3.2 设计难点分析与解决15

3.3 系统概要设计16

3.3.1 系统功能模块图16

3.3.2 系统功能流程图17

3.3.3 系统部署视图18

3.4 本章小结18

第4章 系统详细设计与实现19

4.1 开发环境19

4.2 区块链基本架构19

4.2.1 P2P 网络19

4.2.2 签名和验证22

4.2.3 用户模块23

4.2.4 交易25

4.2.5 区块数据同步28

4.2.6 DPoS 机制30

4.3 文本信息基本管理32

4.3.1 加密信息数据结构32

4.3.2 加解密过程33

4.3.3 信息发送34

4.4 前端模块概述35

4.5 本章小结36

第5章 测试37

5.1 测试原理37

5.1.1 基本概念37

5.1.2 测试工具38

5.1.3 搭建测试环境及测试示例38

5.1.4 测试流程39

5.2 测试结果与总结40

5.2.1 测试内容及结果..... 40

5.2.2 测试结果总结..... 43

5.3 本章小结43

第6章 总结与展望44

致 谢45

参考文献.....46

第1章 绪论

1.1 研究背景

1.1.1 技术背景

随着科技不断进步,信息传递方式也在不断升级,价值的传递方式也同步得到了发展。从电报、电话,再到互联网,人们已经全面进入信息时代。互联网使整个通信网络实现了信息的高效传递,但是却缺乏有效保护信息安全的内在机制,人们还无法点对点地传递带有所有权的信息,在网络上拷贝、传播甚至篡改一条信息的成本几乎为零。一些传统行业因此受到很大冲击,尽管各国政府对网络信息版权保护力度不断加大,仍然难以从技术层面上杜绝侵权的问题^[1]。

在信息时代,信息就是金钱,信息与价值密不可分。人们既然有了互联网这个全球范围的高速稳定的信息传输网络,必然会要求一个与之匹配的高效可靠的价值传输系统,实现去中心化的价值信息交换方式。

区块链(Blockchain)^[2~15]就是在这样的背景下诞生的,它为数字世界里的价值承载和转移,提供了一种基于密码学、分布式系统和点对点网络通信的解决方案,改变了传统的寄希望于中心服务中介的信任方式,仅仅依靠经过数学论证可靠的签名算法,就能解决信用和安全问题。区块链的出现并不是偶然,而是科技发展和人们对计算机系统的要求不断提高的必然产物。

1.1.2 应用背景

随着互联网的蓬勃发展,全球互动在互联网的助力下也越来越紧密,伴随而来的是巨大的信任鸿沟。人们存放在任何一个中心化平台上的信息,都有可能被窃取,并且无法保证永久存储。

例如人们生活中面临的诸如社交系统、银行系统、会员系统等账号及密码信息,它们涉及到个人隐私和财产安全,属于非常重要的敏感信息。各个平台所使用的账号密码不同,密码等级不同,并且会不定时修改,完全依靠头脑记忆把这些复杂的信息存储起来不太现实。一般信息存放在断网的存储设备或者纸质文档上是最安全的,但这种做法极不方便取用,传统的信息载体随时可能损坏,尤其对于纸质文档来说,修改起来更是比较繁琐。在中心化应用里面,管理员会帮助用户保护私密信息,但也有可能监守自盗,在用户不知情的情况下转售这些信息。传统的信息云存储服务一般是非自治的、信息未经加密的,节点数量和空间都很有限,且可靠性无法保证。对于普通用户而言,其安全防范意识和防范技术都很薄弱,如何安置这些重要的文本信息,是他们不得不考虑但又

难以抉择的问题。用户迫切需要一个安全可靠的去中心化应用，来管理这些私密文本信息。

本文就是以这样的需求为设计目标，研究区块链的技术细节及实现方法，利用区块链技术设计一款加密信息备份系统，帮助用户备份自己的私密信息。该系统能通过具有同样需求的用户自行参与维护，贡献自己的存储空间，互相协作存储这些重要的文本信息；这些信息将遍布全网用户节点，每个节点都具有完整的数据备份，不必担心丢失；该系统通过密码学原理，防止被其他用户窥视和篡改，在技术上解决该系统安全可信的问题；这些信息的操作记录是可追溯的，方便用户查看历史记录。当然除了文本信息外，可能还有音频、图片、视频和 PPT 等富媒体信息，也可以采用类似的方法在区块链网络中备份起来，但需要注意解决文件压缩和网络性能的问题。

1.2 国内外研究现状

自 2009 年比特币 (Bitcoin) [16,17] 诞生以来，国内外学者就对其核心的区块链技术产生了浓厚的兴趣。近年来，区块链技术的应用价值也被全球诸多知名科技公司及国家机构嗅探到，纷纷加入到区块链的研发中，加速了区块链技术的不断升级和创新。

2012 年以来，区块链创业领域融资额超过 100 亿人民币。摩根士丹利、富国银行、高盛、汇丰银行、荷兰国际集团 (ING)、法国外贸银行、加拿大丰业银行、中国平安集团等四十多家全球顶级金融机构组成 R3 区块链联盟，致力于研究和发现区块链技术在金融业的应用，并组织其中 40 家金融机构成员进行了一次大规模国际间区块链金融技术测试。IBM、微软、亚马逊等巨头纷纷推出自己的区块链及服务平台，他们也是 R3 区块链联盟的技术支持厂商。2016 年 7 月，英国政府认定首批区块链公共服务技术供应商，并计划在社会福利体系中使用区块链技术。2016 年 7 月荷兰央行宣布为其金融系统引入区块链技术。2016 年 2 月，全球首家专注网络空间基础设施创新的中关村区块链产业联盟在北京成立，这是一个旨在推动区块链行业发展和标准化的非盈利 NGO (Non Government Organization, 即非政府组织) 组织。2016 年 8 月，工信部联合蚂蚁金服、万向控股、微众银行和平安保险等编写《中国区块链技术和应用白皮书》[18]，开展区块链技术和应用发展趋势研究工作。2016 年 6 月 1 日，中国互联网金融协会成立区块链研究工作组。2016 年 12 月，区块链技术首次被列入到了我国《“十三五”国家信息化规划》中，与此同时，贵州省发布了《贵阳区块链发展和应用》白皮书[19]，从政府角度详细阐述了对区块链的认识、态度和发展思路。

区块链的各种社区力量在推进区块链发展上步调达成一致，正积极努力构建一个开放自由、健康有序的区块链生态，区块链的应用场景和商业模式也不断被挖掘和创新。政府开始引导并推动区块链发展，以一系列的政策信号与行动，给区块链刻上一座崭新的里程碑。区块链技术正积极改变当今互联网模式，区块链的发展和落地可能

比想象中要快。

目前,区块链技术已经进入全新的 2.0 时代^[15],实现了智能合约功能,拥有了执行代码的能力,形成了一个由所有节点参与运算的没有国界、没有中心、没有极限的智能计算机系统。因此,最新的区块链研究除了突出其去中心化、分布式存储和不可篡改等特性,还具有价值知晓、区块知晓、图灵完备等特点。其中,图灵完备(Turing complete)是指能计算出每个图灵可计算函数(Turing-computable function)的计算机系统,简单说就是能实现一台通用计算机的所有指令类型,区块链 2.0 能够使用变量,并能执行判断、循环等所有类型指令。因此,区块链 2.0 支持智能合约,可实现各种应用;具有更快的交易速度,最快到秒级交易确认;具有更低的资源消耗,支持 POS (Proof Of Stake)、DPoS (Delegate Proof of Stake) 等低消耗共识机制。

区块链发展非常迅速,在科技界广泛参与和推动下,区块链 3.0 时代也不再遥远。科技界对区块链的发展有很多的奇思妙想,区块链技术的实现方式和技术架构也没有完全成形,其未来有无限种可能性,区块链或许也不再被称作“区块链”,但这项技术本身将会给人类科技领域带来长足的进步。

信息加密技术为区块链提供了安全保障,区块链的成功与加密技术的发展密切相关。只有使用安全高效的信息加密技术,才使得区块链系统被认为是安全可靠的。加密算法主要分为对称加密算法和非对称加密算法,二者各有优点,同时缺点也很明显。对称加密算法具有高速、高效和便于实现的特点,但不便于密钥分配和管理。非对称加密算法密钥分发和管理简单,但速度比较慢。而如果将二者结合起来,使用对称加密算法加密报文数据,采用非对称加密算法生成数字签名,则既能保障数据在网络传输过程中的安全性,又完成了数字签名。目前,对称加密算法推荐采用 AES (Advanced Encryption Standard, 即高级加密标准) 算法^[22,23],非对称加密算法推荐使用 ECC (Elliptic Curve Cryptograph, 即椭圆曲线加密算法) 算法^[24-29],基于 AES 和 ECC 的混合密码体制可以有效提高信息传输的安全性和高效性,这在大量的研究中得到验证,如文献[30-32]。

1.3 研究的目 的及意义

区块链是一项新兴的技术,值得学习和研究。它通过签名授权技术解决了社会上普遍存在的信任问题,通过构建一个安全可靠的分布式记账系统,降低了全社会价值转移和运营的成本,在公证防伪、智能合约、物联网、数据挖掘、身份验证、资产交易、社交通讯、银行结算等领域都逐见成效,体现出了去中心化的优势。

区块链作为一种技术架构,其基本原理并不难理解,但要设计和实现一款完整的区块链应用却很复杂。需要解决用户数据安全问题、设计数据同步及验证过程、设计共识机制等,具体的开发语言、技术框架和实现方式多种多样,业务逻辑繁琐,对编码要求较高。

本文通过该系统的设计与实现，可以发掘性能更好、更利于使用场景的区块链技术实现；通过对区块链技术的应用创新，有助于探索更多的区块链应用方向，扩展其研究领域；通过整合并细化其技术分支，推进技术发展，改善用户体验。对区块链技术及应用的研究和设计，有助于总结去中心化产品的开发经验，有助于学习和利用先进的技术思想，构建去中心化、自制的计算机系统，解决全球互信问题。

本文旨在研究区块链技术的基本理论和技术细节，设计和开发基于区块链的加密信息备份系统，解决生活中的实际问题。该研究有助于解决用户痛点，为区块链的技术创新和应用创新提供新思路，具有非常重要的研究价值和应用价值。

1.4 研究内容概述

本文主要研究区块链的技术原理及其核心思想，设计一套基于区块链的加密信息备份系统，并采用 Node.js 及其相关组件完成该系统的开发与测试。因此，本文将完成以下工作：

- (1) 研究区块链技术核心内容，设计基于 P2P 网络的分布式数据存储方案；
- (2) 研究椭圆曲线系列算法和对称加解密算法，设计用户信息安全保障方案；
- (3) 设计加密信息备份系统，并基于 Node.js 技术实现；
- (4) 完成自动化测试，分析实验结果；

本文重点在于剖析技术实现细节，通过研究、设计并实现该加密信息备份系统，论证区块链技术在去中心的文本信息管理上的技术可行性和安全可靠性能。

1.5 论文章节安排

本文共由六个章节组成，章节与章节之间相对独立，每一章节各小节联系紧密。

第一章是绪论部分，主要介绍区块链技术的背景和国内外研究现状，选题的目的和意义，以及本文的主要研究内容概述。

第二章是关键技术的研究，包括区块链技术概述、密码学技术原理、P2P 及分布式存储原理、共识机制概述等，以及 Node.js 技术介绍。

第三章描述基于区块链的加密信息备份系统的设计目标、难点分析和概要设计。

第四章主要介绍该系统的详细设计和实现过程，详细描述开发要点及流程。

第五章主要介绍该系统的测试原理，并介绍了测试内容及结果。

最后，在第六章部分总结全文并展望未来。

1.6 本章小结

本章对该课题研究的背景、国内外研究概况、课题研究的目的及意义的总概述。对研究的内容、意义及目标做了相关介绍，并列出了本文的章节安排。

第2章 关键技术研究

2.1 区块链基本概念

区块链是以比特币为代表的加密货币的底层技术。它是利用块链式数据结构（一种按照时间顺序将数据区块以顺序相连的方式组合成的一种链式数据结构）来验证与存储数据、利用分布式节点共识算法来生成和更新数据、利用密码学的方法保证数据传输和访问的安全、利用由自动化脚本代码组成的智能合约来编程和操作数据的一种全新的分布式基础架构与计算范式^[4]。

区块链具备如下特性：1）分布存储：区块链采用基于 P2P 网络的分布式存储，通过使用某种同步算法，保证区块链数据在各个节点完整统一；2）公开透明：区块链的每个节点都保存了一个数据备份，并实时获取最新数据。区块链数据可以被任意检索和查询，其本身是没有被加密的；3）无法篡改：区块链上的数据是无法篡改的。要想修改区块链的数据，必须通过验证，要想通过验证，必须修改整个区块链，需要付出巨大的代价。这是对加密技术的巧用，区块之间根据时间顺序链式存储，并实现被数学证明可靠的验证功能，确保无法篡改；4）方便追溯：区块链中的每一笔交易都通过密码学方法与相邻两个区块串联，可以从任一区块往前追溯，并能通过区块查到与之关联的全部交易，悉知任何一笔交易的前世今生；5）存在分叉：在 P2P 网络的环境下，各节点的运行环境和配置是不同的，所有节点各自为政，分叉是无法根本杜绝而必然产生的。只要所有节点按照遵循统一的规则认定一条主链，舍弃不合法的分叉，分叉问题就能得到解决。

区块链数据里最重要的几个概念是地址（Address）、交易（Transaction）、区块（Block）和网络^[1]。地址是指用户的公钥，它是用户的唯一标识，用来指代一笔交易的支出方和接收方。如果说整个区块链是一个账本，那么这个账本上记录的就是一笔笔由一些地址转移到另外一些地址的资产交易。交易是包含价值转移的数据，所有的交易最终被记录到统一的账本上，交易的有效性验证是通过区块的确认来完成的，被确认的区块所包含的交易才能作为合法的交易最终存储在区块链上，无法篡改。区块是打包交易的块结构数据，每产生一个新区块，都会被打上时间戳，并生成按照时间先后顺序排序并加以记录的电子交易证明。每个独立节点之间通过 P2P 网络建立联系，这样就组成了一个去中心化、分布式的电子交易记录时间戳服务器系统。

时间戳服务器是一种基于 PKI（Public Key Infrastructure，即公钥密码基础设施）^[33]技术的时间戳权威系统，它采用精确的时间源、高强度高标准的安全机制来确认系统处理的数据在某一时间的存在性和相关操作的相对时间顺序，为信息系统的防抵赖措施提供基础服务。区块链通过构造这样一个分布式时间戳服务器，达成有效共识，就

可以解决双重支付问题。只要诚实合法的节点所控制的权益总和，大于攻击者节点所控制的权益总和，整个系统就是安全的。而且参与系统的节点越多，节点的性能越强，越能保证系统的安全性和可靠性。任何不友好的节点想要控制整个区块链数据，都将付出巨大的代价。一旦一笔交易被确认写入区块链数据库中，将无法被篡改，所以区块链上的信息都是真实可靠的。区块链技术解决了第一代互联网不够安全可信的缺点，因此可以用区块链系统存储、传递和交换任何有价值的信息。

区块链各个机制及模块有机地结合，使得这样一个去中心化的记账体系成为可能。由于区块链系统的代码和社区都是开源的，关注或使用这个系统的用户都将最大程度的保护区块链系统的健康发展，诚实节点的数量不断增长，节点性能不断提高，使得区块链系统越来越稳定、安全、可靠。区块链的生态体系可以简单的分为协议层、扩展层和应用层，它们相互独立又不可分割，共同协作提供良好的去中心化服务。以下是对各层职能的相关介绍，如图 2-1 是区块链系统架构示意图。

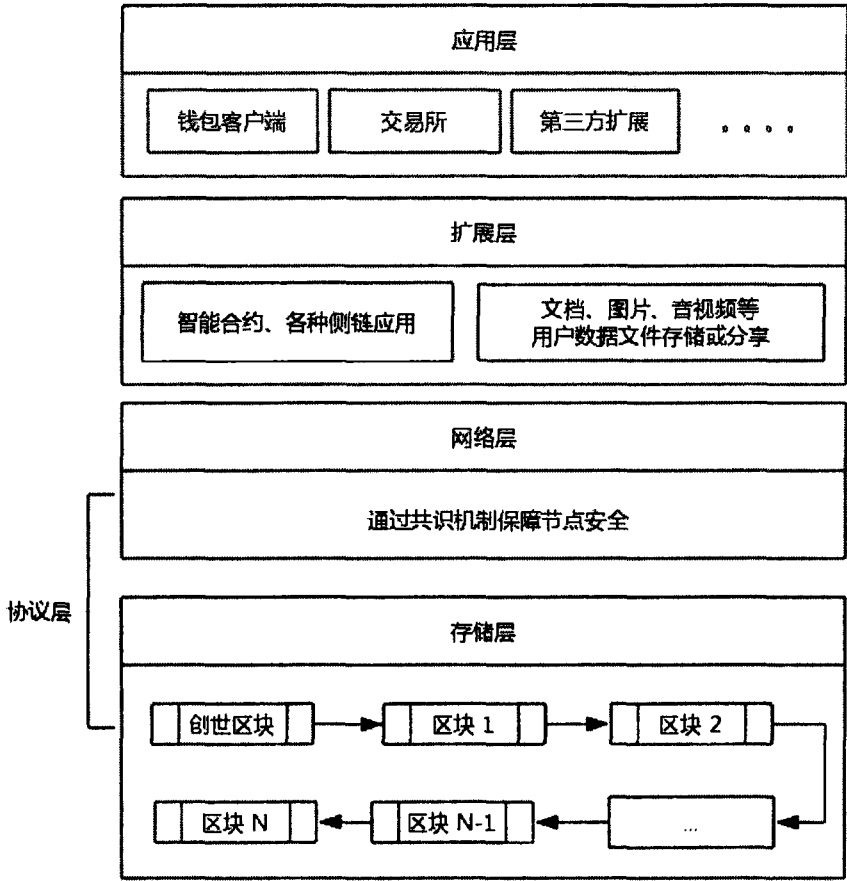


图 2-1 区块链系统架构示意图

(1) 协议层

协议层是区块链生态中的底层架构，其实就是指区块链产品本身，它提供所有网络节点和区块链的核心功能，对外提供 API 给其他层调用。该层主要包括网络编程、分布式算法、签名验证、数据存储技术和共识机制等。

（2）扩展层

扩展层相当于计算机的驱动软件部分，通常是为了让区块链产品更加实用而开发的辅助程序。该层通常分为两类，一类是各交易市场，这是法币和加密货币互相兑换的重要渠道；另外一类是针对某个方向的扩展实现，如智能合约。

（3）应用层

应用层是普通人可以直接使用的区块链产品，即建立在区块链上的丰富多样的生态系统，它是保证区块链应用被广泛接受的终极武器。

区块链适用于大部分传统的中心化应用场景，比如分布式财务管理、电子商务、知识产权保护、分布式存储、福彩、物联网等，并能解决传统中心化应用中无法解决的问题，尤其对于金融行业而言，基于区块链系统的可追溯性，可以很大程度上降低资金的清算和审计成本。

区块链对大部分人来说还是一个新鲜事物，人们对它的认识还存在很多的误区。首先，区块链不是数据库，而是一种旨在解决多方合作与全球互信问题的技术架构，即便它也可以作为数据库来存储源数据，单从存储数据的角度上来说，区块链的效率并不高，因此一般不推荐把大量的原始数据放在区块链上存储。其次，区块链是在多项已有技术的基础上建立起来的新概念，在解决多方合作与全球互信的问题上为现有技术提供新的解决方案。区块链与现有技术是一脉相承的，不可能彻底颠覆现有计算机技术和商业模式而独立存在。在未来很长时间内，区块链技术的适用场景仍需要摸索，它将配合现有的计算机技术，共同建立安全可靠的去中心化应用生态。

“区块链”作为从加密货币中衍生出来的技术名词，它更加形象化、技术化和去政治化地概括了加密货币的底层技术。区块链巧妙地回避了人们对加密货币的争论，很值得作为一项技术去学习、研究和推广，将其扩展到更多带价值信息存储和转移的去中心化应用场景，用新的技术思想，解决基本的信任问题。

2.2 密码学技术概述

2.2.1 公钥私钥体系

现代加密算法的典型组件包括：加解密算法、公钥（publicKey）、私钥（privateKey）。加密时通过加密算法和公钥对明文进行加密，获得密文；解密时通过解密算法和私钥，对密文进行解密，获得明文。根据公私钥是否一致，可分为对称加密算法和非对称加密算法。这两种模式分别适用于不同的使用场景，形成互补关系，很多时候可以根据业务实际情况组合使用，形成组合机制。

2.2.1.1 对称加密

对称加密是指公钥和私钥相同的公私钥加解密模式，该模式优点是加解密速度较快，占用的空间小，保密性良好，强度高。它的缺点很明显，参与加密的多方都需要持有同一密钥，一旦某人泄露这个密钥，安全性就被破坏，如何分发密钥成为主要问题。它适用于批量数据的加解密，但不适用于需要签名的场景。

AES (Advanced Encryption Standard, 即高级加密标准) 系列算法是安全性较高的对称加密算法，具有密钥灵活性、较高的可实现性、较高的安全性能及实现效率，其密钥建立时间极短，且灵敏性良好。在长文本信息的对称加密中，推荐使用 AES 系列算法。

2.2.1.2 非对称加密

非对称加密与对称加密正好相反，是公钥和私钥不同的加解密模式。其公钥一般是对外公开的，任何人都可以获得，私钥一般只有用户个人持有和保护，具有私密性，不能被他人获取。该模式的公钥和私钥是分开的，比较容易管理，且容易完成密钥的分发。同时，公私钥对是唯一有效的，想要碰撞出任何一个公钥的私钥或私钥的公钥都很难。其缺点是加解密速度慢，一般适用于签名场景或密钥协商的场景，在大量数据的加解密场景下并不适用。非对称加密算法的代表包括：RSA^[34]、ElGamal^[35]、ECC (Elliptic Curve Cryptograph, 即椭圆曲线加密算法) 等。目前，RSA 算法已经被认为是不够安全的，现主要推荐使用椭圆曲线系列算法来完成。

2.2.2 椭圆曲线系列算法

椭圆曲线指的是由韦尔斯特拉斯 (Weierstrass) 方程 $y^2+a_1xy+a_3y=x^3+a_2x^2+a_4x+a_6$ 所确定的平面曲线。若 F 是一个域， $a_i \in F, i=1,2,\dots,6$ 。满足该等式的数偶 (x,y) 称为 F 域上的椭圆曲线 E 的点。 F 域可以是有理数域，还可以是有限域 $GF(P)$ 。椭圆曲线通常用 E 表示。除了曲线 E 的所有点外，尚需加上一个叫做无穷远点的特殊点 O 。

在椭圆曲线加密中，利用了某种特殊形式的椭圆曲线，即定义在有限域上的椭圆曲线。其方程为： $y^2=x^3+ax+b(\text{mod } p)$ 。这里 p 是素数， a 和 b 为两个小于 p 的非负整数，它们满足： $4a^3+27b^2(\text{mod } p) \neq 0$ 。其中， $x,y,a,b \in F_p$ ，则满足式 (2) 的点 (x,y) 和一个无穷点 O 就组成了椭圆曲线 E 。

椭圆曲线离散对数问题 (ECDLP)^[37] 定义如下：给定素数 p 和椭圆曲线 E ，对 $Q=kP$ ，在已知 P,Q 的情况下求出小于 p 的正整数 k 。可以证明，已知 k 和 P 计算 Q 比较容易，而由 Q 和 P 计算 k 则比较困难，至今没有有效的方法来解决这个问题。这就是 ECC 的原理所在。

ECC 是代替其他非对称加密算法强有力的竞争者。ECC 的安全性能更高，160 位 ECC 可以与 1024 位 RSA、DSA (Digital Signature Algorithm, 即数字签名算法) [36] 达到相同的安全强度。ECC 的计算量小，处理速度快，在加解密和签名验证等操作上，ECC 算法对私钥的处理速度远比 RSA、DSA 快很多。另外，ECC 算法的密钥尺寸小得多，使得其占用空间也比其他算法小很多。所以，ECC 算法的安全性高、速度快、带宽要求低，相较其他非对称加密算法而言，具有更加广泛的应用前景。

2.2.2.1 ECDSA

ECDSA (Elliptic Curve Digital Signature Algorithm, 即椭圆曲线数字签名算法) [38] 用于数字签名，是 ECC 与 DSA 的结合，整个签名过程与 DSA 类似，只是签名中采用了 ECC 算法。ECDSA 的核心目标，不在于证明其加密数据的保密性，而在于证明数据是签名者本人发出的，是不可抵赖的，技术上解决了信任问题。

2.2.2.2 ECDH

ECDH (Elliptic Curve Diffie-Hellman) [40] 是基于 ECC 的密钥交换算法。交换双方可以在不共享任何秘密的情况下协商出一个共享密钥 (Shared Key)。ECDH 的定义如下：

假设密钥交换双方为 Alice、Bob，其有共享曲线参数 (椭圆曲线 E 、阶 N 、基点 G)。

(1) Alice 生成随机整数 a ，计算 $A=a*G$ 。Bob 生成随机整数 b ，计算 $B=b*G$ 。

(2) Alice 将 A 传递给 Bob。 A 的传递可以公开，即攻击者可以获取 A 。由于椭圆曲线的离散对数问题是难题，所以攻击者不可以通过 A 、 G 计算出 a 。Bob 将 B 传递给 Alice。同理， B 的传递可以公开。

(3) Bob 收到 Alice 传递的 A ，计算 $Q=b*A$

(4) Alice 收到 Bob 传递的 B ，计算 $Q'=a*B$

Alice、Bob 双方即得 $Q=b*A=b*(a*G)=(b*a)*G=(a*b)*G=a*(b*G)=a*B=Q'$ (交换律和结合律)，即双方得到一致的密钥 Q 。

2.2.2.3 ECIES

ECIES (Elliptic Curve Integrated Encryption System, 即椭圆曲线集成加密方案) [39] 是一种公钥加密方案，包含公钥操作、对称加密算法、消息认证码和哈希计算，机密性好。由于公钥加密比对称加密算法要慢许多，公钥加密算法主要用在加密少量关键数据上，而长消息一般由对称加密算法来加密。

2.3 P2P 网络和分布式存储

2.3.1 P2P 网络

区块链采用了基于 P2P (Peer-to-peer)^[41~43]的网络架构。P2P 是指位于同一网络中的每台计算机都是彼此公平、对等的,各个节点共同提供网络服务,不存在任何特殊(中心)节点。区块链中每个节点之间互相连接组成了区块链网络,保证了整个区块链系统的安全。

区块链网络中的节点相关功能包括:1)每个节点都会收到交易被广播到全网的消息,将新交易收集到节点内存,并组装成区块;2)每个节点都尝试获取创建区块的资格,然后把合法有效的区块数据在全网进行广播;3)当且仅当验证了该区块中的交易都是合法有效的,并验证了工作量证明以后,各个节点就认同该区块的有效性,并在区块链的末尾写入新的区块,整个区块链才得以不断延长;4)所有节点都默认将最长链作为唯一正确的区块链主链,并在此基础上持续完成上述延长区块链的过程^[1]。

实际上,新交易的广播不需要抵达网络中的全部节点,只要交易信息能够抵达足够多的节点,它们将很快被整合进一个新的区块中。节点对被丢弃的信息进行了容错处理:如果一个节点没有收到某特定区块,那么该节点将会发现自己缺失了该区块,就会向较长链的节点发出下载该缺失区块的请求。

在 P2P 网络中,每个节点都具有高度的自治性,节点之间可以互相自由连接。节点不具备强制性的中心控制功能,任何一个节点都可能成为阶段性的中心。节点与节点之间会通过网络形成非线性因果关系,不会因为某些节点的错误而导致整个区块链系统崩溃。这种开放式、扁平化、平等性的系统现象或结构就被称之为去中心化^[44,45]。这种省略掉中心的点对点网络,取代了传统的中心服务器,使得参与区块链系统的每个节点都是独立的主机,将所有数据都同步在由全网节点组成的云系统中,形成一个证明与自证的系统。

2.3.2 分布式存储

传统的网络存储系统采用了集中存储的方式存放数据,中心化存储服务器的存储空间非常有限,不能满足存储大规模数据的需求。而分布式存储系统采用可扩展的系统结构,将数据分散存储在多个独立的设备上,提高了系统的可靠性、可用性、可扩展性。

中心化的存储方式或多或少面临着信息安全和永久存储的问题,而基于区块链的分布式云存储将是解决这一问题的最佳方案。与目前中心化提供的云存储空间不同,基于区块链技术的分布式云存储不但可以储存,还可以同时证明这份数据是真实可信

的，并且永远不被修改，可以说是加密后分散式的云存储。

基于区块链的分布式存储还具有非常实用的特点和功能：1) 实现了碎片资源的可利用，每个人都可以通过分享个人的硬盘空间获得金钱回报；2) 大众可以广泛参与，所有人都可以访问公开区块链上的数据，所有人都可以发出交易等待被写入区块链，共识机制过程的参与者通过密码学技术以及内建的经济激励维护数据库的安全；3) 该存储方式还可以高效、低成本运行，区块链技术在网络上公开、透明、开源的，不需要通过任何机构和组织，可以随时随地上传、下载所需要的信息，比起购买昂贵的存储设备及配套的人力来说，更加经济、实惠；4) 它还具有较高的安全性，传统的云存储公司购买或租用服务器来存储客户文件，同时使用 RAID (Redundant Arrays of Independent Disks, 即磁盘阵列)^[46]方案或多数据中心的方法来保护数据的安全性。而使用区块链技术则不需要中心，不需要购买昂贵的设备及维护人力。区块链技术让文件存储在一个分布式、虚拟和分散的网络中，这样就不需要像传统的云存储那样依靠硬件的维护来保证存储的可靠性^[2]。

2.3.3 二者的关系

区块链采用了基于 P2P 的分布式存储方式，P2P 为分布式存储提供了网络支持，但仍需要复杂的分布式同步算法作为理论基础。基于区块链的分布式存储，核心思想在于使得网络上存在完全相同数据的 N 个备份，当其中一个备份的数据发生改变，数据能够快速同步到其他的网络节点^[15]。

普通的分布式存储系统中存储的数据，通常很难验证其来源和真实性。而在区块链系统中，数据的状态和来源可以一并保存，人们不仅可以验证某一条数据最后一次是由谁添加以及被添加的时间，还可以验证这条数据是否被非法篡改。所有节点都参与同步和验证数据，保证数据备份的完整性，任何节点恶意修改数据，都不会影响到整个网络数据的正确性。

2.4 共识机制

在任何一个去中心化体系结构中，由于参与系统的各个节点地位平等，当决策出现分歧时，需要达成一个合理的共识，来维护区块链系统的健康稳定。共识机制作为区块链核心技术之一，实现了一系列激励用户参与和维护系统的策略算法，它不仅可以在节点间的合作出现问题时解决冲突，还可以鼓励用户参与到区块链系统的维护工作中，提高系统的活跃度和稳定性。共识机制在社区的推动下不断被创新，使得区块链的应用领域变得更广，促进了区块链生态系统的健全发展，以下是区块链常用的共识机制的介绍。

2.4.1 POW

POW (Proof Of Work), 即工作量证明机制, 这里的工作量不是指人力上的工作, 而是指计算资源的投入, 即算力投入。各节点通过与或运算, 计算出一个满足约定难度的随机字符串, 以此来获得本轮记账权, 打包新的交易数据, 在全网其他节点验证后一起存储, 并获得区块奖励, 该过程在加密货币中叫做挖矿。在 POW 机制下, 每个节点都可以公平地参与竞争, 并通过激励机制构建一个积极循环的生态系统, 从而逐渐积累起保护系统安全的庞大算力。算力难度的增加使得激励的成本逐渐提高, 加密货币的价格也就随之上涨。POW 的优点是完全去中心化, 节点是自由进出的; 缺点是计算出约定的结果会浪费大量的计算资源, 并且达成共识的周期相对较长, 不太适合商业应用。

2.4.2 POS

POS (Proof Of Stake, 即股权证明), 就是直接证明你持有的份额, 它是 POW 机制的一种改进的共识机制, 把 POW 中的算力改成了系统权益, 拥有较大权益的节点就有较大的概率成为下一个广播报块的节点。具体地根据各个节点所持有加密货币的比例及时间, 等比例降低挖矿难度, 从而加快找随机数的速度。POS 没有挖矿过程, 而是在创世区块内就写明了股权证明, 之后的股权证明只能被转让, 不能挖矿。POS 虽然在一定程度上缩短了达成共识的时间周期, 但本质上并没有解决商业应用的痛点。

2.4.3 DPoS

DPoS (Delegate Proof of Stake, 即股权授权证明机制) 是一种新的保障加密货币安全的共识算法, 其实质是给持股人一把可以开启他们所持股份对应表决权的钥匙, 而不是给他们一把能挖矿的铲子。其基本特点是持股人永远掌控大局, 持股人数量较多且具有一定的代表性, 这样系统便是去中心化的。DPoS 优点在于, 将持股人的盈利最大化, 最小化维护网络安全费用, 将网络的效能最大化, 最小化网络运行的成本 (如带宽、CPU 等), 这样大幅缩小参与验证和记账节点的数量, 减少了等待确认时间, 同时也就减少了达成共识的时间。不过整个共识机制还是离不开代币, 而某些商业应用不需要代币的存在。

2.4.4 Pool

Pool (验证池机制) 是在传统的分布式一致性算法上加上数据验证技术的共识机制, 目前在行业链中大范围使用。该机制的优点是不需要代币也可以工作, 在成熟的分布式一致性算法基础上, 实现了秒级的共识验证。其缺点是去中心化程度不高, 更适用于多方参与的多中心商业模式。

2.5 Node.js 技术概述

区块链系统的开发对语言没有特殊要求。其中，分布式算法虽然复杂，但基本上属于业务逻辑，实现上不用考虑语言优势；对于加解密技术、签名验证技术和数据库技术，基本上所有编程语言都有相关的可以直接使用的库函数或扩展包，这些模块在开发语言上的差别不大。点对点网络的实现^[50]和并发处理是区块链系统开发的主要难点，因此网络编程能力强、支持并发的平台通常比较适用于开发区块链协议层。Node.js 技术除了满足上述要求，还具有学习成本低、项目构建灵活、社区资源丰富等特点，是开发区块链应用不错的选择。

Node.js 是一个基于 Google Chrome V8 引擎的 JavaScript 运行环境，主要用于方便敏捷地搭建响应速度快、易于扩展的网络应用。V8 引擎本身使用了一些最新的编译技术，极大提升了 Javascript 这类脚本语言代码的运行速度。Node.js 使用了一个事件驱动、非阻塞式 I/O 的模型，对一些特殊用例进行了优化，同时为文件系统、数据库之类的资源提供了替代的接口，以可扩展的方式简化了对慢资源的访问，使其在非浏览器环境下运行得更好^[48]。

Node.js 实现了开发者们在服务器与浏览器客户端之间共享开发语言及代码的美好愿望，并能提供非常好的响应速度和兼容性，现主要被用于编写服务端代码，直观易懂，轻量又高效。采用 Node.js 开发，可以享受到诸多好处：1) Node.js 具有模块化组织代码的能力，组织代码非常方便；2) Node.js 的出现，让 javascript 的第三方组件像雨后春笋一样遍地开花，资源非常丰富，可以节省很多基础功能的开发成本。Node.js 的包管理器 NPM (Node Package Manage)，是全球最大的开源库生态系统，一条 npm 安装命令就可以完成常用扩展包的安装，开发者只需要在程序里直接引用，不需要手动重复造轮子，非常方便；3) Node.js 具有非常不错的全栈处理能力。项目开发完成后，通常还有很多后续工作需要处理，比如对前端代码 (js 或 css) 文件进行合并压缩，以及项目部署和系统测试等，Node.js 都很容易做到。总之，Node.js 具有不错的运行性能和比较敏捷的开发流程，不用学习多种语言就能处理全栈应用，是当今主流的网络应用开发平台之一^[49]。

2.6 本章小结

本章主要从技术角度，介绍了区块链技术的概念、特点及发展前景，围绕区块链技术简要介绍了相关的密码学知识、P2P 网络特点、分布式云存储特征和共识机制四个主要技术分支，围绕本系统的技术实现简单介绍了 Node.js 的技术特点。

第3章 系统分析与概要设计

3.1 系统设计目标

在 Node.js 环境下,搭建区块链基本框架,在此基础上,设计并开发加密信息备份系统。基于区块链的特性,用户通过使用该系统将享受到以下便利:

(1) 用户可以将一些简单的私密文本信息存放在该系统中,区块链网络的节点众多,每个节点都保存了完整的数据备份,大量节点同时损坏的概率几乎为零;

(2) 用户可以在任意节点登录自己的账户,提取和查看相关信息,这些信息以密文的形式在互联网中传播;

(3) 用户可以将文本信息通过该系统安全地发送给其他账户;

(4) 用户可以选择将自己的私钥加密后备份到该系统中,以免丢失。

该系统的设计与开发具体要求如下:

(1) 利用区块链的技术思想,构建去中心化的加密信息备份系统;

(2) 该系统主要基于 Unix 环境开发,最终完成一个可远程访问的分布式节点程序,各服务节点提供 API 接口,以便节点之间通信和前端模块调用;

(3) 该系统对运行性能没有要求,以实现技术架构和应用功能为主。

综上所述,用户只需要管理一个账号,即可在任意时间和地点,提取到有价值的私密信息。当然,用户可以通过区块链存放任何形式的数据,本文着重研究文本信息的加密存储,其他形式的信息可以参考类似的方式,交给去中心的分布式系统来管理。

3.2 设计难点分析与解决

该系统的设计和开发,主要存在如下几个难点。

(1) 共识机制的设计

在区块链系统中,不同的应用对共识算法的选择有所不同。该系统没有挖矿过程(如比特币挖矿),用户对系统维护的参与度不高,但中心化程度要求较高,必须保证一定数量的优质服务节点,所以该系统采用基于 DPoS 的共识机制。在设计和开发过程中,需要理清受托人产生过程和区块创建资格的指定过程,以及每个奖励周期、块周期和投票周期需要完成的操作等。为了保证受托人的服务质量,需要对受托人进行信用评估,为用户的投票提供依据,对于服务质量不好的节点可以投反对票,促使该节点降级。每一个小模块都有着比较复杂的业务流程,设计和编码难度较大,开发前需

要整理好思路并写明设计文档，以免遗漏重点步骤。

（2）区块数据更新

该部分主要难点在于如何验证缺失的区块，区块数据的加载过程，加载后如何验证以及如何解决区块分叉等问题。对于这些问题，主要参考其他区块链应用的区块数据同步和验证过程，运用到本系统中，设计并实现相关的具体流程。

（3）交易数据扩展

早期基于区块链的数字货币应用，只是将区块链作为记账本记录一些转账信息，然后打包存储起来。而现在，需要存放不同形式的交易数据，方便用户分类查询并提取出来，能清晰的看到信息的变更记录。该部分需要设计合理的数据表结构，使得交易与扩展信息关联记起来，在代码中以 `asset` 字段展现在交易里。

（4）便捷性设计

区块链的概念对于一般的用户而言还比较陌生，接受起来也比较困难，需要简化操作流程，改善用户体验，降低学习成本，让用户比较自然地接受区块链技术的相关概念，从传统的中心化应用中平稳过来。用户可以像传统的应用系统一样，通过账号和密码登录系统，并管理相关信息。对于用户公私钥的管理，文本信息的加解密，数据的同步和验证等，都由系统自动完成并保障数据安全，用户不需要了解技术细节。

当然该系统的设计和开发还有许多未知的难点，可以借助社区的力量和检索工具，总结更多的解决方案，逐一攻克难题，完成各个细节部分的分析、设计和实现。

3.3 系统概要设计

3.3.1 系统功能模块图

该系统主要包含以下模块：P2P 网络模块、交易模块、区块模块、用户模块、共识机制模块、加密信息管理模块。P2P 网络模块包含节点创建、网络连接；交易模块主要包含交易的生成、交易信息的扩展；区块模块主要包含区块的创建、区块的验证和同步；用户系统模块包含用户注册登录、用户公私钥的管理；共识机制模块主要包含奖励周期和委托周期；加密信息管理模块包含用户文本基本管理、文本加解密过程和文本信息发送功能。该系统各模块及模块的功能结构如图 3-1 所示。其中，交易模块是区块链的核心功能之一，在该系统中，交易记录了用户在该系统中有价值的操作，但不一定有代币的转移。

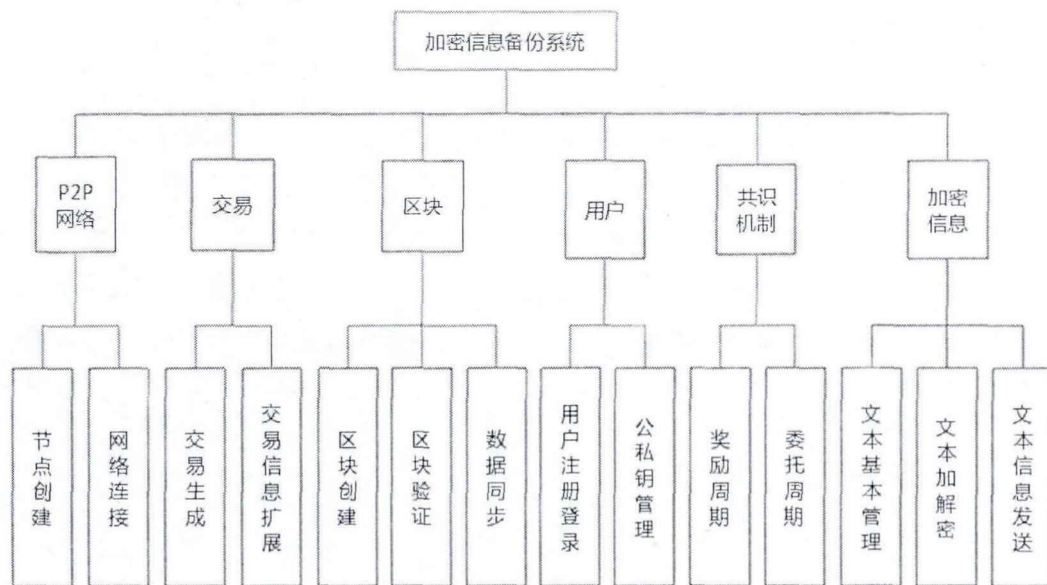


图 3-1 系统功能模块图

3.3.2 系统功能流程图

该系统对普通用户可见的功能非常简单。用户在完成注册和登录后，主要完成一些常规的信息添加、编辑、删除等管理工作 and 信息发送操作。基于该系统架构，还可以实现其他去中心化的扩展功能，比如投票。如图 3-2 是该系统的功能流程图。系统底层会为用户的不同操作，产生相应的交易数据，交易的确认和打包，区块数据的创建和同步，以及交易的扩展功能等，都由系统节点独立自动完成。功能性交易在验证了合法性以后，在各自节点完成相应操作。所以对于开发者而言，该系统的功能模块众多，业务流程复杂，但对用户而言，这些过程是不可见的。

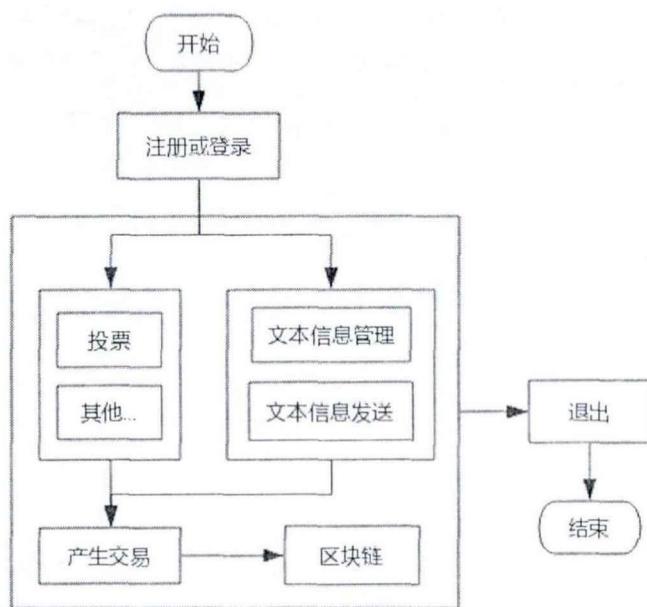


图 3-2 系统功能流程图

3.3.3 系统部署视图

本系统的部署非常简单。在系统开发完成后，配置一台基本满足运行要求的服务器作为初始节点，然后通过用户创建的节点连接初始节点或其他用户节点，完成 P2P 网络连接，基本形成多节点网络后，初始节点就和普通节点一样，可以随时删除，不再影响整个系统的正常运行。如此，就留下完全由用户自己组建的区块链网络，每个节点保留一份数据备份，每个数据备份都记录了当前在线的节点信息，如图 3-3 是该系统的部署视图。

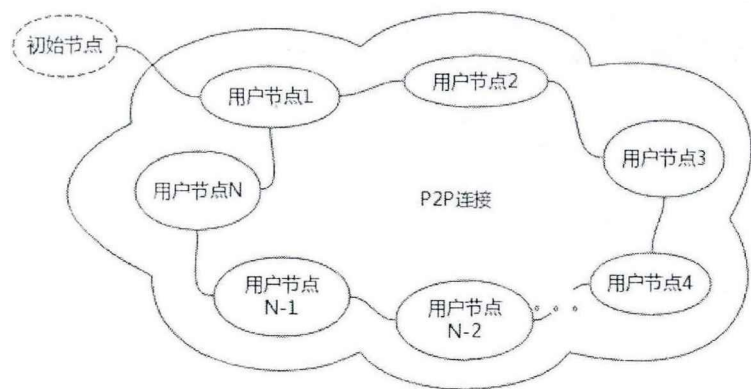


图 3-3 系统部署视图

3.4 本章小结

本章主要介绍了基于区块链的加密信息备份系统的设计目标，然后对设计难点进行了分析并提出解决方案，最后介绍了系统的概要设计，为详细设计和系统实现做理论调研及开发准备。

第4章 系统详细设计与实现

4.1 开发环境

该系统基于 Unix 环境开发，所有环境配置和安装都是以命令的方式完成，环境搭建简单。详细步骤如下：

(1) 安装 Node.js

首先用 curl 命令安装 nvm (Node Version Manage)，nvm 是 node 的版本管理器，用它可以在很方便的管理和切换 node 版本，最新版 nvm 的安装命令详见 <https://github.com/creationix/nvm>。

然后用 nvm 安装 Node.js，终端执行：nvm install node (version)。

最后安装 cnpm，cnpm 是 npm 的国内镜像，可以提高 Node 组件在国内环境下的加载速度。执行如下命令即可：

```
npm install -g cnpm --registry=https://registry.npm.taobao.org。
```

(2) 安装 sqlite 数据库

执行安装命令：sudo apt-get install sqlite。

(3) 创建项目

在新创建的项目文件夹，执行命令创建初始化项目：npm init。根据命令行提示，填写项目相关信息，及完成项目的初始化。然后添加入口代码文件 (app.js) 和静态资源文件 (html、css 等)。利用 npm 工具安装必要的组件并引用，如 express、gulp 等，执行命令：cnpm install (component) --save。组件安装完成后，只需要执行 node app.js 命令即可运行，后续开发在此基础上一步步构建即可。

4.2 区块链基本架构

4.2.1 P2P 网络

4.2.1.1 路由扩展

路由 (Router) 是网络应用的入口，通过解析 URL (Uniform Resource Locator) 地址，完成系统逻辑处理的请求分发，实现相应的业务功能。几乎所有的 Web 应用都需要实现路由功能，以便提供 API 处理客户端请求，该系统的路由功能采用 Express 组件实现。Express 是基于 http 协议的 npm 组件，它封装了 Node.js 的 connect 底层模块，使得路由设计简单而灵活。

以下是路由的示例代码（router.js），首先加载 cors 模块设置跨域，允许任何客户端调用。然后配置地址映射，用于映射路由和函数方法的对应关系。

```
var cors = require('cors');

var Peer = require('./modules/peers.js');

...

module.exports = function (app) {
  app.use(cors());
  app.get('/api/peers', Peer.getPeers);
  ...
}
```

4.2.1.2 节点的构建过程

利用路由映射方法，可以定义一些节点的读取接口，定位到相关的 Peer 类操作方法，然后执行处理用户请求。如图 4-1，是 Peer 类及其关联关系图，从图中可以看出，每个完整节点的数据库都保存了当前运行中的节点信息，包括 IP、网络端口、运行状态和版本等信息，并对外提供查询接口，有助于新节点连接已有节点。

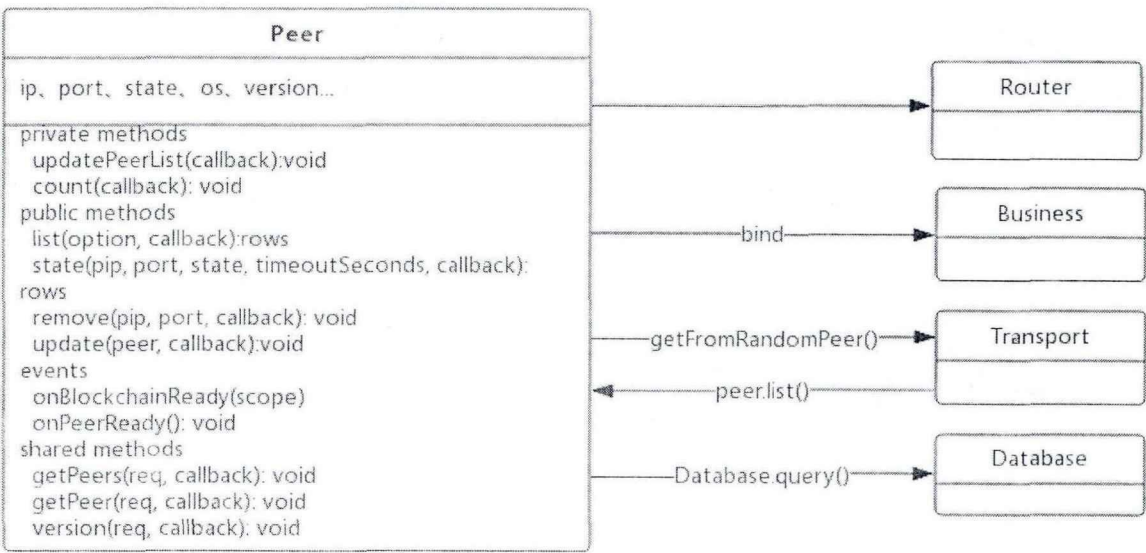


图 4-1 Peer 类及其关联关系图

有了路由模块，节点与节点之间就能完成通信，并执行初始化方法和同步方法，完成新节点保存。具体主要完成如下几个操作。

初始化节点：用户在创建新节点的时候，需要连接已知的节点，加载最新的数据。但想要在网络中搜索出一个该应用的服务节点，可以说是大海捞针。为了让新节点能够同步最新的区块链数据，需要将一些初始节点配置在 config.json 文件中，也可以通过

命令行参数--peers 提供，软件开始执行的时候就会从这些节点择优同步最新的区块链数据，然后在最新数据的基础上开始工作；

写入节点：本系统把节点信息保存在本地数据库的 Peers 表中，当区块链数据加载完成的时候，依次将网络中配置好的节点写入本地数据库，本地已存在的记录就不做操作，然后更新本地节点列表，触发加载完成事件；

更新节点：节点加载完毕后，循环调用 updatePeerList 函数，通过 transport 模块的 getFromRandomPeer 方法，逐个随机的验证节点信息。请求成功后返回该节点的运行状态信息，然后更新数据表中与该节点对应的信息。如果请求失败，说明该节点当前没有提供服务，就将该节点从数据表中删除。

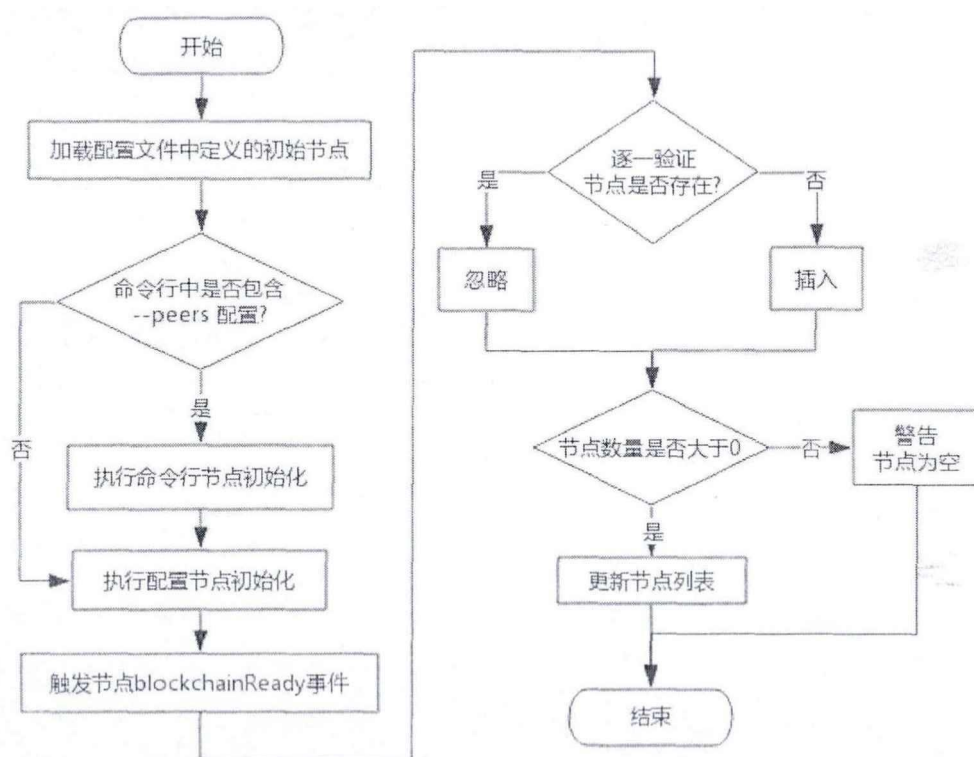


图 4-2 Peer 类主要函数调用流程图

如图 4-2 是该过程的完整流程图，按照以上方法构造的 P2P 网络有如下特点。

第一，提供了一定数量初始节点及列表，保证初始节点快速完成创建和本地更新，使新创建的节点不至于成为孤立节点。

第二，节点可以自由跨域访问，保证了节点之间的通信稳定。

第三，节点具备自我更新能力，通过定期更新节点状态，并淘汰停止工作的节点，保证整个网络的通畅。

当系统节点达到一定数量，就会形成一个互联互通、高效传输的不死网络。理论上随着节点的增加，系统会越来越稳定。节点间的通信，将在该 P2P 网络的基础上进行。

4.2.2 签名和验证

区块链应用之所以是无需监管、安全可信的，主要原因就在于采用了经数学证明安全可靠的签名验证算法对数据进行签名和验证，使得区块链中的所有操作都是用户本人的行为。这里所指的签名和验证，主要是对交易数据进行签名，对交易合法性进行验证，保证传输过程不被篡改，收到的信息经验证真实有效后才被写入区块链。

签名具有简单、安全和可验证的特点，作用是确定交易的所有者，用加密技术代替了人的笔记，证明是用户本人所为，是不能抵赖的。该系统签名方法封装在 Signatures 类里，该类提供了两个简单的接口，最核心的方法是 addSignature，用于用户使用自己的私钥对数据进行签名。如图 4-3，是 signature 类图及其关联关系图。

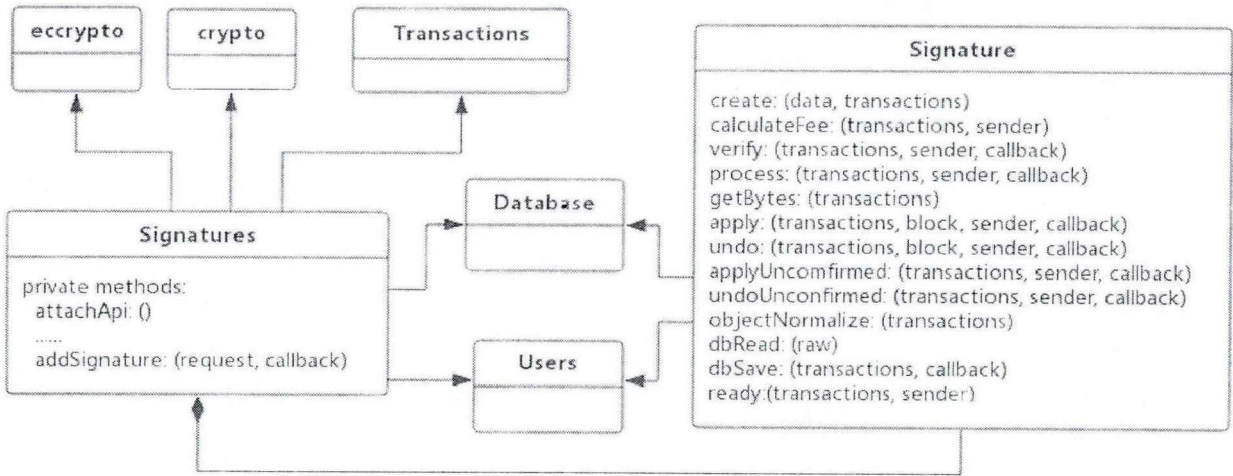


图 4-3 signatures 类图及其关联关系图

公钥对是签名和验证过程中非常重要的部分，是由算法随机生成的，通常不可能重复。其中，公钥作为用户的唯一标识，不仅标识了用户身份，也是作为验证其交易合法性的关键字符；私钥作为用户的身份证明，一定要妥善保管，一旦丢失，用户就无法证明自己。密钥对是几乎无法伪造的，一个公钥只能对应一个私钥，碰撞出某个公钥的私钥或私钥的公钥几乎不可能。通常用户在发起一笔交易的时候，需要用自己的私钥对交易数据进行签名，这样保证了交易信息来自于用户本人，网络在验证交易合法性的时候，通过该用户的公钥进行验证即可。

本系统使用 Crypto 组件^[52]随机生成私钥，并用 Eccrypto 组件^[53]生成该私钥的公钥。生成密钥对的示例代码如下：

```
var crypto = require("crypto");
var eccrypto = require("eccrypto");
var privateKey = crypto.randomBytes(32);
var publicKey = eccrypto.getPublic(privateKey);
```

该系统采用 ECDSA 算法完成交易信息的签名和验证。签名时，先对数据进行 hash256 运算，然后调用 Eccrypto 组件的 sign 方法进行签名，示例代码如下：

```
var msg = crypto.createHash("sha256").update(data).digest();
eccrypto.sign(privateKey, msg).then(function(signature) {
  console.log(signature);
});
```

验证时，调用 Eccrypto 组件的 verify 方法完成交易数据校验，示例代码如下：

```
eccrypto.verify(publicKey, msg, signature).then(function() {
  console.log("Verify success");
}).catch(function() {
  console.log("Verify failed");
});
```

4.2.3 用户模块

该系统的用户模块主要完成账户信息的记录工作，为用户在本系统中的其他操作提供数据支持。用户可以任意创建一个账户，并设置账户名，账户名唯一且不可修改。传统的区块链应用是没有账户名的，该系统中使用账户名作为地址别名，使得普通用户更容易接受，改善了用户体验。

该系统的用户角色其实是一个公钥地址。账户在被创建时，系统会利用上一节介绍的密钥对生成方法自动为其生成唯一对应的公钥和私钥，其中公钥作为用户的唯一识别码存在数据库中，同时也是用户产生交易的地址，私钥由用户自己保存。

用户可以通过用户名和密码登录该系统，当然密码并不是必须的，区块链应用里面最能证明身份的仍然是用户的私钥，密码的作用仅仅是用来备份私钥。私钥主要有两种管理方式：第一，由用户自己保管；第二，用户可以选择使用设定的密码，采用对称加密算法，加密自己的私钥，即可把加密好的私钥存放在系统中，使用时把加密后的私钥加载到本地，本地客户端通过用户密码解密后就获得私钥，只要保证私钥不在网络中传输，那么该用户的账号就是安全的。第二种方法虽然安全性有所降低，但保存密钥更加方便，相当于给私钥也设置了一个别名，便于记忆，不用随身携带。对于普通用户而言，只需要记忆一个账号和密码登录即可完成所有操作，密码可以随时修改，与以往的中心化平台的使用没有什么区别，系统底层已经为其做好了账号安全的保护工作。

如图 4-4 是用户注册流程图，图中详细分解了用户注册时的操作流程。注意，注册操作也是一种交易类型，也需要使用私钥签名，并经过区块链网络确认后注册才会生效，关于交易部分的详细介绍见下一小节。私钥的加密备份过程可以在注册后随时完

成，与注册时的加密私钥备份过程基本一致。

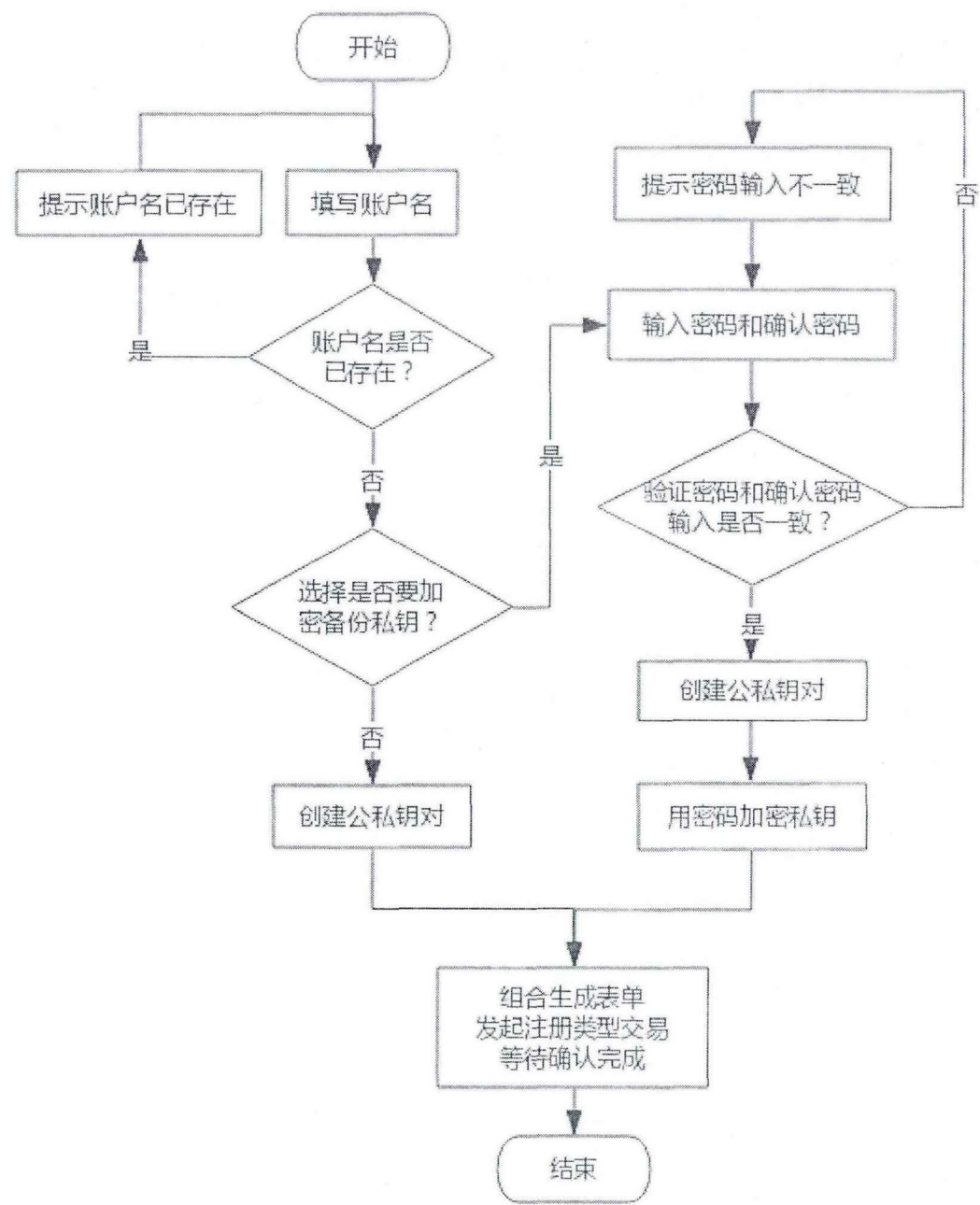


图 4-4 用户注册流程图

该系统的登录过程实质上是本地客户端的初始化过程，并没有严格意义上的与服务器进行密码正确性验证的交互过程。用户只要能提供合法的公私钥，就能在客户端正常使用该系统。如图 4-5 是该系统用户登录的流程图，需要注意的是，如果用户加密备份自己的私钥，可以选择通过密码解出私钥的明文，并验证私钥是否正确。验证私钥是否正确的方法很简单：使用该私钥对任意字符串进行签名，如果使用该账户的公钥能验证通过则该私钥正确，反之则错误；或者用该账户的公钥对任意字符串进行签名，如果使用该私钥验证签名通过则是该私钥是正确的，反之则该私钥是错误的。如果解出来的私钥错误，说明密码不正确，则要求重新输入密码。

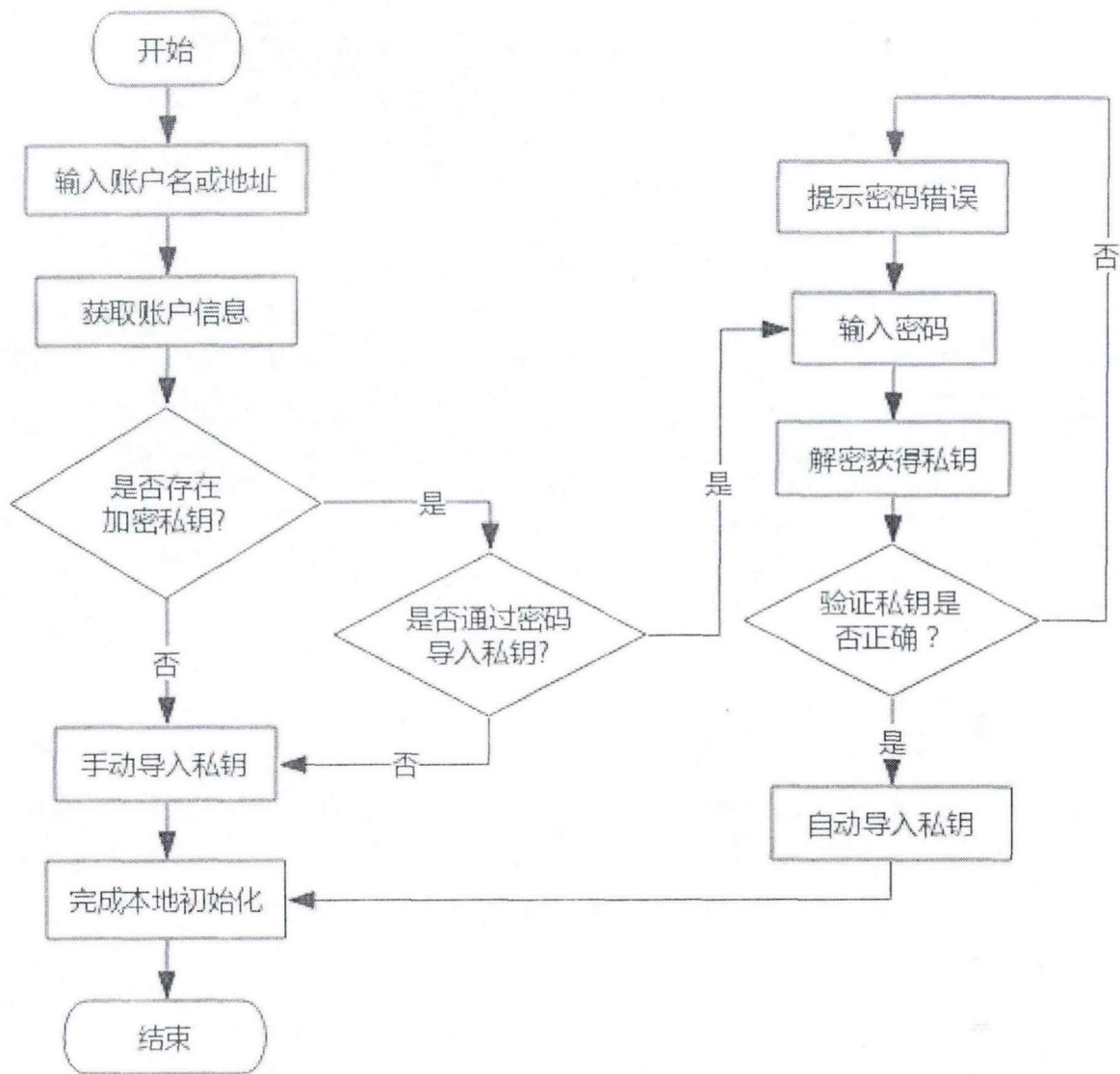


图 4-5 用户登录流程图

4.2.4 交易

区块链中的交易，原指通过区块链网络把加密货币从付款地址转移到收款地址，并将转账记录的数据保存到区块链的过程，其本质是价值所有权的变更，价值的转移只是该过程的结果。在该系统中，交易还可能包括用户的文本信息的变更与发送、账户信息的修改，以及其他业务处理，可能附带有价值转移，所以在该系统中交易的概念更加广泛，可以说是指一切需要验证用户身份的有效操作。交易是区块链应用的核心功能，上述的 P2P 网络模块、签名模块和用户系统模块，都是为了准确生成交易数据，然后迅速广播到其他节点，由全网节点验证其合法性以后写入到区块链这个交易总账本中。图 4-6 是交易类及其关联关系图，从图中可见交易的数据部分相对独立，但业务上与其他模块又关系紧密，所以该部分的开发逻辑性较强，编码具有一定难度，生成一笔交易具体包括以下几个过程。

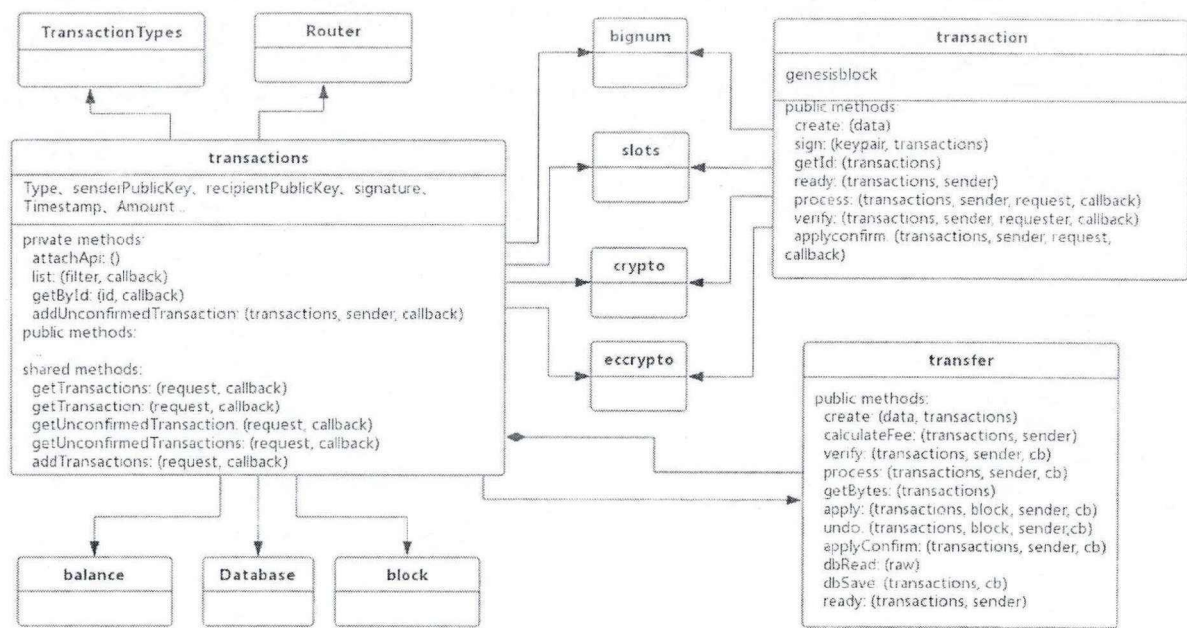


图 4-6 交易类及其关联关系图

（1）生成交易数据

交易数据主要包含交易的类型（Type）、支付账号、接受账号、交易数量（Amount）、时间戳（Timestamp）和有效签名等信息，而且是不含任何私密信息的合法交易数据。其中，支付账号和接受账号其实就是交易双方的公钥地址，如果是功能性的交易，则不需要填写接受账号。交易数据里要记录交易产生时的时间戳，便于追溯。交易数据示例如下：

```
{
  "type": 0,
  "amount": 10,
  "fee": 0,
  "timestamp": 1488361707320,
  "recipientPublicKey": "0477826520dc5c6bed2***8c37c3b3df558190aa6999",
  "senderPublicKey": "04cf6cab3f268153991e63***1e6397bad2588b0e65246",
  "signature": "d04ad75***956404",
  "id": "5534571***943011068"
}
```

这些数据虽然有的是由系统自动生成，不需要用户输入，但为了防止数据冗余和攻击者伪造虚假数据，每个字段都需要进行类型校验和逻辑验证，该过程是不可或缺的。具体的数据校验过程主要涉及到验证发起交易的账户是否存在、收款地址是否合法、余额是否充足等，都需要逐个检验。在扩展的交易里面，还可能涉及到扩展数据

的校验。如图 4-7，是生成交易数据流程图：

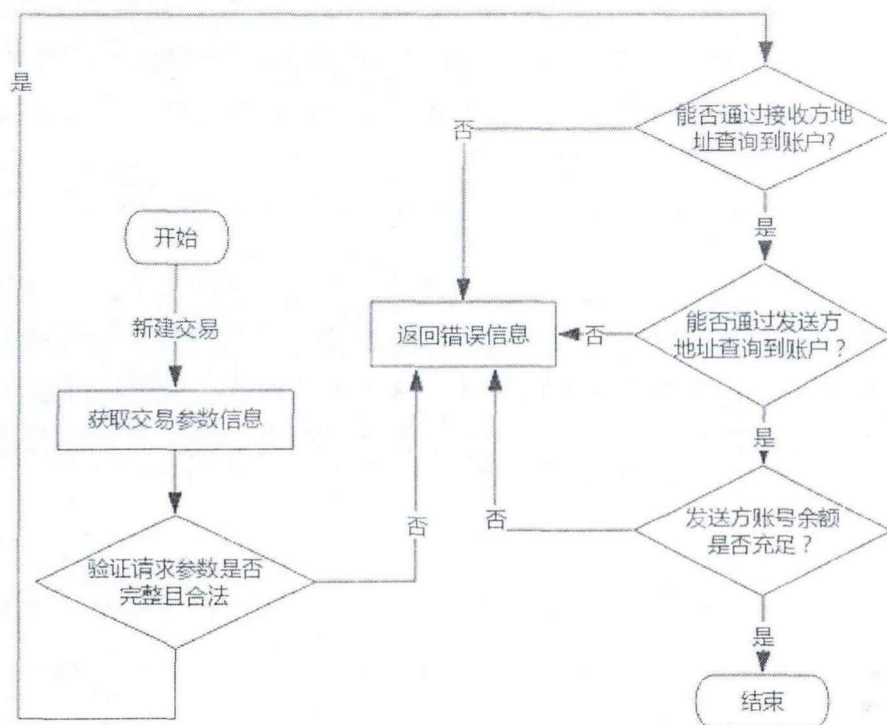


图 4-7 生成交易数据流程图

(2) 给交易签名

即便验证了交易请求信息基本正确、生成了基本的交易数据还不够，还需要对交易数据做进一步处理。其中包括使用发送方的私钥对交易数据进行签名，目的是为了标识交易所有权。只有用发送方的私钥签名，那么在区块链网络验证交易时，使用发送方的公钥验证才能通过，以此证明该笔交易是由发送方本人发出。签名方法与本章第三节所介绍的方法一致，签名前通过 `getBytes` 方法获取交易数据的字节数组。

最后，交易数据里还应该生成交易的唯一 ID，这个 ID 受交易数据的影响，能够标识交易记录的特征，与数据本身对应。实现时将交易数据进行 `hash256` 运算，然后把得到结果按照规则进行字符串压缩，最终生成包含了交易及加密信息的字符串，而非自增型索引字段。生成交易 ID 的示例代码如下，其中 `bignum` 是大整形数据处理组件。

```
Transaction.prototype.getId = function (transaction) {
    var hash = crypto.createHash('sha256').update(this.getBytes(transaction)).digest();
    var temp = new Buffer(8);
    for (var i = 0; i < 8; i++) {
        temp[i] = hash[7 - i];
    }
    var id = bignum.fromBuffer(temp).toString();
    return id;
};
```

（3）验证交易合法性

通常情况下一笔交易在经过6到10个区块验证后，就基本完成了确认，该笔交易将再也无法更改。因为这时候想要拒绝、变更交易的难度已经非常大，几乎不可能完成。每笔交易在广播到区块链网络之前，必须验证交易的合法性，不合法的交易是没有机会广播到网络的。验证过程包括使用发送方的公钥验证签名，确保数据完整性，再验证基本信息是否正确，还要保证待验证的交易是未经确认的，同时也不是用户重复提交的交易。

交易的合法性验证过程是在一个节点上独立完成的，交易经验证合法后就可以直接将其加入到区块链了。任何节点收到新的交易信息时，都需要重新进行验证，以至于任何网络攻击行为都只会影响一个节点，保证整个网络始终是安全可靠的，使得安全性大大提高。

（4）广播到网络

区块链系统没有中心化的服务器和数据库，必须借助 P2P 网络把交易数据同步到其他节点，该过程保证交易数据具有大量的备份，无法篡改，并且可以轻松查询追溯。

交易的广播，不需要到达每个节点，只要保证一定数量的节点接收到即可。广播交易的具体过程，主要是获取当前未确认的交易，在本地验证通过后，发出广播指令（broadcast），该指令中包含广播节点的数量。实现时，随机选取指定节点数量的节点，逐一发送 POST 请求，要求这些节点完成上述的未确认交易的合法性验证，同时更新自己的节点信息。当多个节点验证该笔交易通过后，该交易就完成了确认，价值转移过程也就实现了，交易双方的余额也会发生相应的变化。

（5）写入区块链的过程

广播交易的过程，还伴随着区块的产生和交易打包过程，最终写入到区块链。写入区块链的过程涉及到比较复杂的区块数据创建、广播和同步等操作，将在下一节详细介绍。

4.2.5 区块数据同步

从数据库的角度去理解，区块数据其实就是一张“自引用”的数据表，但又和一般意义上的数据表有所区别。首先，该 Blocks 表上的每条数据都记录着前一条记录的信息，从任何一条记录开始都可以往前顺序追溯，直到第一条记录。同时，该系统中的区块数据表使用的是经过加密处理的信息字段作为外键，具有签名认证作用。这样设计不仅能使区块数据顺序增长，避免数据重复，还能实现自我认证，防止区块数据被篡改。

交易与区块形成多对一的关联方式，这样的设计方便追溯。区块数据是有序自引用的，只要搜索到区块，那么该区块对应的交易数据就被查询到。交易数据在区块链系

统中是非常庞大的，把交易打包到区块中逻辑存储起来，有助于缩小索引范围，使得交易数据的查询效率大大提高。如图 4-8 是区块链数据表及其关联关系图。

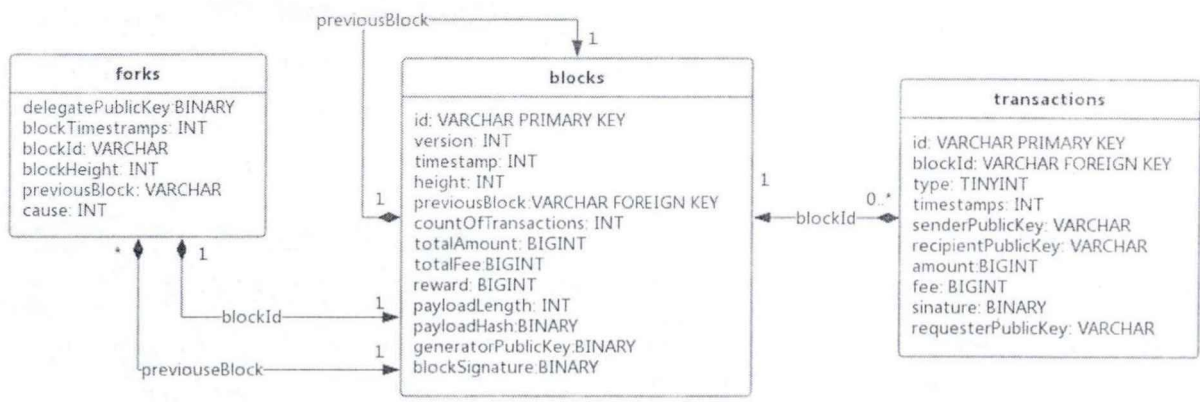


图 4-8 区块链数据表及其关联关系图

在实际设计和开发过程中，该模块将完成如下几个步骤：

(1) 保存创世区块

每个节点在最初运行的时候，会创建一个创世区块，此后以该区块作为区块链的根，同步后续区块数据。每个节点都配置了完全一致的创世区块，如果不一致，就无法加载到正确完整的区块链主链。如果没有配置或创建创世区块，则需要在网络上找到合法的主链，追溯其创世区块，在本地保存。

(2) 加载和验证本地区块

该过程也是软件初始化过程的一部分，为了确保数据没有被篡改，都需要先加载并验证本地区块。加载方法被放在入口文件中去执行，通过 loadBlockChain 方法来加载。该方法主要循环调用 loadBlocks 方法，按照索引顺序逐一加载，当区块链数据同步完成后，再触发 blockchainReady 事件，执行被加载区块的验证方法。loadBlocks 方法会传入 offset 参数，用来指定一次性加载区块的数量。

验证本地区块的过程，主要是由节点验证新收到的区块中打包的交易是否合法，该过程需要对加载的区块逐个进行验证。对于每个区块的验证，首先追溯其前序区块，验证是否在主链上，然后验证区块的签名是否正确，再校验区块时段是否正确，最后按照上一节的方法，逐一验证区块中的每一条交易数据是否合法。

(3) 创建新区块

当其他区块数据加载并验证完成之后，就可以创建新的区块。新区块是周期性创建的，这个周期被称为块周期。一个块周期的时长是根据交易数据的大小和产生频率经验性指定的，一般一个区块的交易数据最多在 1MB 大小即可。为了保证所有的交易能被打包到区块，进行正常的确认，块容量和块周期的值也会做相应的调整。

每次创建新区块都会获取并再次验证未确认交易，放入一个数组变量里暂存起来以

备打包成区块。通过 Create 方法创建新区块，该方法传入一个对象，该对象包括 privateKey、timestamp、previousBlock、transactions 几个部分。其中 privateKey 是报块节点的私钥，timestamp 是块时段时间戳，previousBlock 是当前最新区块、transactions 是未确认交易数组。该 Create 方法利用这些参数信息，计算区块奖励、交易总额、区块数据大小、区块 ID 等信息，以及使用报块方的私钥对区块数据进行签名。签名的方法和前文一致，ID 的计算和交易类似。

新创建的区块很快被广播到网络中，经由其他节点完成上述验证过程。实际上，不是每个节点都有资格创建新区块，需要在新区块中提供股权证明，该部分参见下一节中对 DPoS 机制的详细介绍。

（4）产生区块链分叉

理想的情况下，自引用表结构的数据是不存在分叉的。但区块链是基于 P2P 网络的分布式存储，每个节点各自独立维护本地数据，实际物理情况复杂多样，同步起来非常麻烦，加上人为干预，使得区块数据分叉的问题无法避免。只要正视这个矛盾，设计解决分叉的机制，就能保证只存在一条合法的主链。

区块链分叉在新区块写入区块链的时候产生，如果新写入的区块被判断为分叉的块则需要另向“forks”表插入对应数据，注意代码中需要判断分叉的原因。分叉的原因包括以下情况：1) 高度和父块区块相同，两者的前序区块不同；2) 受托人时段不同；3) 交易已存在；4) 高度和父块相同，ID 不同。这里父块是指该节点上最后一个区块。

（5）同步区块，并解决分叉

同步区块和解决分叉是同时进行的，一旦产生分叉，说明本地区块链存在偏离主链的风险，该节点就会主动触发该过程。具体实现时，在网络中随机获取远程节点的正常区块链高度，如果本地区块链高度小于该节点正常高度，则从该节点同步区块数据。并将分叉的区块查询出来，直接删除即可。所有节点都遵循该机制，就保证了整个网络及时确认和更新最新正确的区块数据。

4.2.6 DPoS 机制

共识机制是区块链技术的核心，是分布式去中心化应用软件特有的策略机制。在传统的中心化系统中，系统的共识是由中心决定的，开发难度低。而在去中心化的架构体系中，节点间的互操作，节点行为的统一管理，只能通过共识算法来支撑实现。所以，共识机制的设计是区块链应用开发中无法回避的难题。

DPoS 机制整合了社区的资源，它的去中心化依赖于一定数量的代表，而不是所有的用户。人们为了自身利益的最大化，必然会推举稳定、可靠的优良节点作为委托节点，参与维护区块链交易数据，使得系统更加安全和去中心化。

该系统中的 DPoS 机制主要包括以下过程。

（1）竞选受托人

DpoS 机制中的股东具有一定的代表性或权威性,以此来保证系统的去中心化和服务质量。受托人需要完成注册相关信息,提供高质量节点,并接受投票,票数最靠前的受托人才能算是正式的受托人。受托人数量根据系统要求的中心化程度调整,数量越多,中心化程度越高。该系统定义了一种投票的交易类型,任何普通用户都可以参与投票,由此来产生新一轮的受托人。投票过程是周期性的,所以被叫做投票周期,受托人每轮都会变更,确保不良节点会被及时剔除掉。

投票是通过交易来实现的,整个网络都具有投票记录的备份,所以投票的记录也是不可篡改的,投票产生的结果是可靠和有效的。每一个委托周期,系统会生成一个投票结果,存放在数据库中,投票周期表(rounds)与受托人表(delegates)是一对多关系,因此可以很方便定位到受托人:块周期一般等于块高度,也可以通过时间戳计算出来,根据块周期和受托人数量可以定位到投票周期,进而定位到受托人,所以块高度和受托人公钥是一一对应的,据此可以验证区块的公钥是否正确。相应的原理可以用如下伪代码表示:

```
current_delegate = active_delegates[slot % active_delegates.size]
```

该系统还将对竞选过程做以下两点改进:1)如果出现受托人破坏网络正常运行的行为,用户在投票时可以对其投反对票。如果该受托人的反对票达到一定数量,就会强制该不良节点降级,避免其对网络的破坏可能;2)用户可以在使用节点的过程中,对节点的服务质量进行评价,并提出反馈意见,形成信用体系。利用这些节点的信用信息,可以帮助用户筛选高质量节点。

（2）循环产生新区块

块周期是其他周期的基础,为了减小数据库的 IO 操作,块周期会根据用户数量及使用情况做相应的调整。在每个块周期,都对应了一个委托人具有创建新区块的资格,受托人每产生一个新区块都会得到一定数额的代币奖励,每个受托人都有机会。奖励额度将逐年递减,最后保持在一个较低的水平,代币少量增发,激励受托人认真严肃参与,维持系统稳定运行。

周期的产生,是通过 loop 方法实现的。该方法每隔一段时间执行一次,每次执行都先判断当前时间在哪个块周期,如果当前块与最新区块高度一致,说明系统已完成该区块创建。反之,则判断该节点是否具有报块资格,如果有该资格才进行区块创建和广播工作。loop 方法的执行周期一般取 1 分钟以上,应该比块周期小很多,这样就能实现在每个块周期都会创建出新区块,并在全网广播数据,实现数据同步。

4.3 文本信息基本管理

以上便是本系统所设计的区块链核心框架，有了这个框架，使得云端存储信息变得异常方便，基于区块链的 P2P 分布式存储的设想才能实现。下面将对该系统进行扩展，完成文本信息的存储，实现用户重要信息的加密备份管理。类似于本文所述的加密信息，其他扩展数据也可以用类似的方式，对价值信息进行分布式存储。

4.3.1 加密信息数据结构

如图 4-9 是 articles 表结构及其关联关系图。其中，articles 表存放文本基本信息，用 enable 字段标识逻辑存在性，address 字段来关联用户，title 和 desc 是加密后的文本信息标题和正文；article_records 数据表表示 article 的操作记录，它是类型为 Article 的交易数据的扩展信息。ArticleRecord 只是在操作 Article 后提交交易时，存储的 Article 的快照，提现在交易扩展信息，与交易数据一一对应。这样就可以通过 article 对应的 article_records 数据，来查看用户对某一条文本记录操作的历史状态。

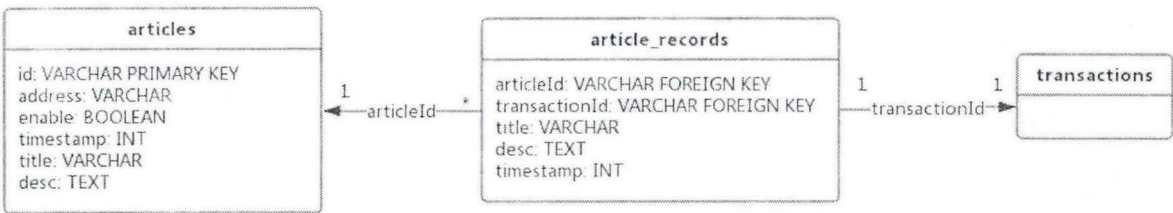


图 4-9 Article 数据表结构及其关联关系图

如图 4-10 是 Article 类及其关联关系图。用户对文本的操作，其实都是诸如 createArticle、updateArticle、deleteArticle 等类型的交易。每一次提交都会创建一条 article_record 数据，用来存储交易扩展信息。其中 title 和 desc 是加密后的字段，各节点在完成交易合法性校验后，执行数据表 articles 的增、改操作。

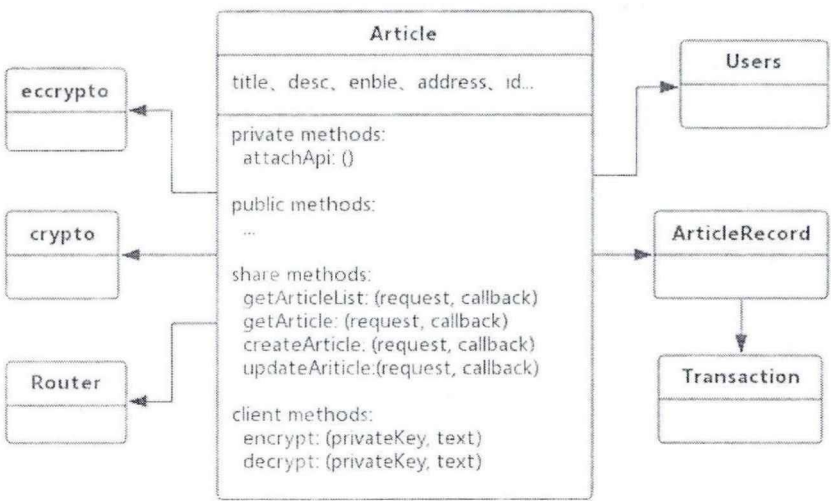


图 4-10 Article 类及其关联关系图

这些数据在网络中的同步过程，比如发起交易、网络确认等，都是通过前面实现的区块链框架来完成的。在删除操作中，文本信息不会被彻底删除，只是在 articles 中用 enable 字段来区分，方便追溯。

4.3.2 加解密过程

加解密过程由客户端软件完成的，明文信息不会在网络中传播。该过程对用户是不可见的，用户可以像常规信息发布平台一样管理文本信息。

对于加解密技术的选择，业界的通则是使用现成的被认可的加解密组件，严格按照开发文档去做，这也是使用加解密技术最安全的方式。Node.js 的 Crypto 模块，用于 HTTPS 网络或 HTTP 连接，也对 OpenSSL 的 Hash、HMAC、加密、解密、签名和验证等方法进行了封装。本系统的文本加解密就采用 Crypto 的 aes192 对称加密算法，利用用户的私钥进行加解密，加密方法示例代码如下：

```
function encrypt (key, textBuffer){  
    var encrypted = "";  
    var cip = crypto.createCipher('aes192', key);  
    encrypted += cip.update(textBuffer, 'utf8', 'hex');  
    encrypted += cip.final('hex');  
    return encrypted;  
}
```

解密方法示例代码如下：

```
function decrypt(key, encrypted){  
    var decrypted = "";  
    var decipher = crypto.createDecipher('aes192', key);  
    decrypted += decipher.update(encrypted, 'hex', 'utf8');  
    decrypted += decipher.final('utf8');  
    return decrypted;  
}
```

加解密技术是该系统中非常重要的技术基础，就实现上来讲，单纯的文本加解密并不困难，选择更加安全高效的加密方法更加重要，它是区块链应用确保安全可靠的基石。利用区块链在解决信用问题上的优势，将加密后的信息备份在区块链上，用户不用担心这些信息丢失，也不用担心信息被其他人看到。

4.3.3 信息发送

该系统除了可以完成加密文本信息的备份，还可以实现加密文本信息的在网络中的安全发送，即把文本信息由发送方转移到接收方，并能保证文本信息不会被第三个人看见。

如图 4-11，是该文本信息发送过程的示意图。具体实现时，发送方需要发起一笔类型为“SendArticle”的交易，用发送方的私钥和接收方的公钥协商出共享密钥对文本数据进行 aes192 对称加密，将密文存放在该交易扩展数据中，并对交易进行签名。该交易和其他交易一样很快会被区块链确认，确认后该交易完成。接收方验证交易时，发现有文本消息发送过来，确认数据来源后选择接收，提取交易扩展里的文本信息，用接收方的私钥和发送方的公钥协商出共享密钥对密文解密，解出明文后完成图中的生成备份操作，即由接收方产生新的文本数据，使用接收方的私钥对文本进行 aes192 对称加密，最后存储起来即可。

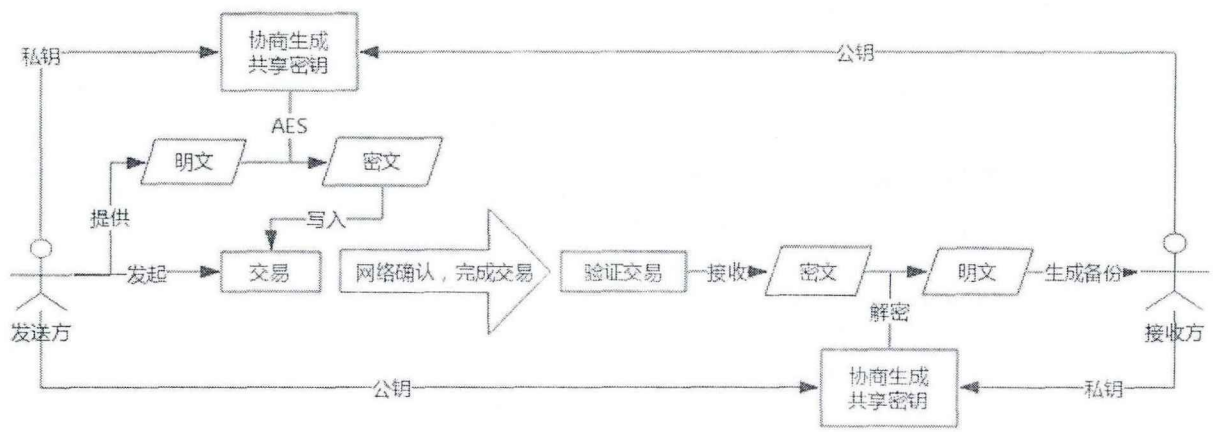


图 4-11 文本信息发送过程示意图

其中，对称加密算法采用 crypto 组件的 aes192 算法。共享密钥的协商过程，采用 eccrypto 组件的 ECDH 模块实现，密钥协商的示例代码如下，可以验证 sharedKey1 和 sharedKey2 完全相同。

```
eccrypto.derive(sender.privateKey, recipient.publicKey).then(function(sharedKey1) {
    console.log(sharedKey1);
});

eccrypto.derive(recipient.privateKey, sender.publicKey).then(function(sharedKey2) {
    console.log(sharedKey2);
});
```

如此，就实现了信息的发送操作，整个过程可以保证文本信息的安全，文本明文仅对交易双方可见。被转移的信息放在交易扩展中，交易双方都可以用自己的私钥和对

方的公钥协商出共享密钥后查看具体内容，而不需要泄露各自的私钥信息。可见区块链在价值信息管理上的可行性、可靠性和安全性，解决了传统互联网信息存储和发送不够安全可信的问题。

大多数的用户之间信息交换都可以基于该原理，只是在不同的需求下实现细节可能有所不同。比如，有时文本信息发送方不需要查看交易扩展信息，即仅实现文本信息所有权变更，这类需求在业务上是可能存在的，那么在发起交易前，需要对文本信息通过 ECIES 公钥加密算法用接收方公钥加密，接收方确认后用私钥解密并完成备份。这样就能保证该交易里面的文本对发送方不可见，仅对接收方可见。Node.js 的 `eccrypto` 组件提供了 ECIES 算法的实现，使用起来非常简单。ECIES 加密示例代码如下：

```
eccrypto.encrypt(sender.publicKey, text_buffer).then(function(encrypted) {  
  console.log(encrypted)  
});
```

对应的解密示例代码如下：

```
eccrypto.decrypt(recipient.privateKey, encrypted_buffer).then(function(plaintext) {  
  console.log(plaintext.toString());  
});
```

4.4 前端模块概述

该系统的节点服务端除了对外开放 API，提供给客户端调用，还能结合 Node.js 强大的前端构建能力，设计可以供用户远程访问的前端展现模块。

该系统采用前后端分离的 Web 架构，浏览器一次性加载所有静态资源，直接构建一个完整的客户端，服务端不需要针对网页端单独编写一套 API 接口。该架构的特点是只需要加载一次静态资源，此后的用户操作都异步完成，没有复杂的页面跳转，与使用本地客户端的体验几乎完全一致。同时，利用 Gulp 和 Phonegap 工具还可以实现前端模块的本地化构建，使得所有静态资源及 js 脚本打包生成不同平台的客户端，实现跨平台开发。

该系统的前端部分采用基于 Angularjs 的 MVVM 架构的前端框架，当然还有诸如 vue.js、react 等同类框架可以选择，而 Angularjs 在构建大型前端应用上更利于模块化管理。MVVM 架构的前端框架使得数据与 dom 双向绑定，开发者只需要把重心放在数据交互上，前端界面会根据数据或者前端路由变化自动响应，大大提高了开发效率和用户体验。

4.5 本章小结

本章主要基于概要设计和技术调研对该系统进行详细设计，对关键技术做了详细分析，并对技术实现过程做了简要描述。根据区块链系统的开发过程，分别介绍了 P2P 网络的搭建过程、签名与验证的基本原理和实现、用户系统的概述和特点、交易的概念与设计、区块链的数据同步过程和解决分叉的措施以及该系统的 DPoS 共识机制的特点和完成的工作等。最后，基于构建好的区块链系统，设计加密信息的交易扩展，实现了文本信息的加密备份以及文本信息的发送功能。

第5章 测试

5.1 测试原理

5.1.1 基本概念

软件测试是指通过手动操作或者运行测试代码的方式，检验软件是否满足规定的需求和开发要求，弄清开发预期与实际结果之间的差距的过程。同时也是帮助认清软件开发的准确性、完整性和性能的软件研发过程。

(1) 单元测试

单元测试是指开发人员编写的用于检测程序单元代码正确性的一小段测试代码。程序单元指的是软件中的最小可测试部件，即很小的、很明确的功能点，通常是单个程序、函数或过程等，对于面向对象编程，主要是指类方法。单元测试就像是可执行的文档，描述了在各种使用条件下调用代码是，期望该段代码完成的功能，帮助新老开发者充分了解代码的用法。

(2) 集成测试

集成测试是指按照概要设计说明书的要求，验证软件单元在组装成模块、子系统或系统的过程中，各部分工作是否满足相应技术指标和要求的活动，其测试对象是已经通过单元测试的软件单元。集成测试通常是在单元测试完成之后进行，否则测试的最终效果会受到很大影响，程序单元代码的纠错代价也会大大增加。

(3) 系统测试

系统测试是将完成集成测试的软件与整个计算机系统中的其他部分相结合，在实际运行环境下对整个计算机系统进行严格有效的整体性测试。其主要目的是发现软件的潜在问题，保证软件在整个计算机系统下的正常运行，同时保证被测试软件对系统中的其他模块不产生影响。

(4) 性能测试

性能测试是对软件运行性能的评价过程，软件性能是指软件的响应及时度能力、执行效率和使用范围等。性能测试验证了软件开发成果是否满足系统设计要求，为改进软件开发质量提供了重要的参考依据。

(5) 基准测试

基准测试是一种测量和评估软件性能指标的方法，通过运行一系列测试程序，然后把性能计数器的结果保存起来，并在系统环境的描述中进行注解，最后将系统过去和现在的性能表现进行对照比较，就能确认系统或环境的所有变化。

(6) BDD

BDD (Behavior-Driven Development, 即行为驱动开发) 是一种通过自然语言来书写测试用例的方法, 主要特点是非程序员可读, 适用于软件需求方或非软件开发人员的。该方法目的是为了让开发者着重把精力放在编写代码方法上, 最大程度的减小开发语言与商业客户、利益相关者、项目管理者等不同领域语言之间的翻译代价。

5.1.2 测试工具

Node.js 的资源丰富, 社区强大, 因此搭建测试环境也不是什么难事, 该系统测试模块主要用到 Mocha、Should.js、SupperTest、Benchmark 等几个工具。

Mocha 是 Nodejs 里最常用的测试框架, 灵活、简单、有趣, 支持多种 node 的框架测试, 同时支持异步和同步的测试, 支持多种方式导出结果, 也支持直接在浏览器上跑 Javascript 测试代码。使用 Mocha 测试的人很多, 相对比较可靠, 安装也很方便, 运行 npm 安装命令即可。

Should.js 是 Node.js 的测试断言库, 它的主要优点和作用是更加口语化地描述期望。

SupperTest 是基于 Super-agent 的 http 接口测试库, 它提供了更抽象的方法, 支持 http 请求测试, 也支持调用 superagent 的接口方法。

Benchmark 是一个用于 Node.js 和浏览器上的 JavaScript 的基准测试库。

除了上述用来编写和运行测试代码的测试组件, 还可以利用 Postman 等接口调试工具, 对客户端环境进行仿真, 在没有前端展示的情况下测试网络接口。

5.1.3 搭建测试环境及测试示例

首先, 安装所需的测试组件, 通过 npm 包管理工具即可安装完成。测试的代码放在名为 test 文件夹, 然后按照 Node.js 的构建过程构建测试代码即可。测试时, mecha 会自动读取 test 文件夹中的测试代码, 通过 mecha 执行测试脚本代码完成自动化测试。上述测试工具, 使得描述测试用例变得非常简单。

该系统的自动化测试代码主要基于用户操作, 模拟用户发出的客户端请求, 验证系统是否返回正确的结果。同时, 记录请求耗时, 用于分析软件运行性能。尤其是该系统在复杂的网络环境下, 会出现很多意想不到的错误, 需要考虑的因素和覆盖的测试点很多。以下是一段简单的测试代码示例:

```
var supertest = require('supertest');
var request = supertest('http://127.0.0.1:7000');
var should = require('should');
describe('test/peers.test.js', function () {
  it('should return all online enable peers', function (done) {
```

```

request.get('/api/peers').end(function (err, res) {
    should.not.exist(err);
    res.status.should.equal(200);
    res.body.sucesss.should.equal(true);
    res.body.peers.length.should.equal(1);
    done(err);
});
});
});

```

像 `should` 和 `it` 这样的测试描述语句是由 `mocha` 提供的，它帮助测试人员定位到代码位置，并理解测试结果的含义。以上测试代码示例执行时，将会产生如图 5-1 形式的测试结果。

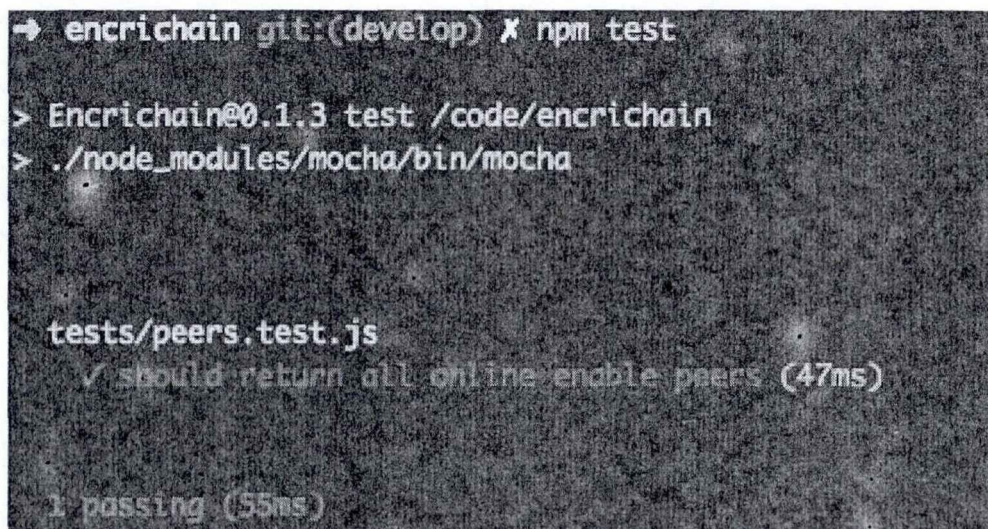


图 5-1 测试示例结果截图

5.1.4 测试流程

由于 Node.js 技术的响应速度和处理速度都较快，该系统除了 I/O 消耗率被硬件配置限定，并可以在算法上面优化以外，系统性能上的瓶颈不多。另外，该系统着重实现功能，验证可行性，实验设备简单，对性能要求也不高。所以，尽管系统的性能测试非常重要，但这里不是该研究重点，暂不做实验和分析。在实际开发过程中，性能测试仍必不可少，这能为后期的改进提供重要的参考依据。

该系统的测试部分主要包含单元测试、集成测试和系统测试。通过编写和执行自动化测试代码，完成接口正确性和响应速度的测试；通过手动操作完成对系统功能的测试，主要是对前端展现的正确性进行测试。如图 5-2 是测试流程图。

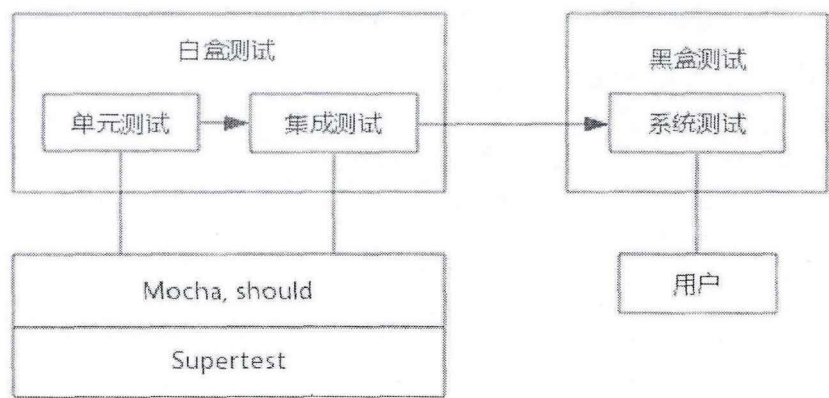


图 5-2 测试流程图

5.2 测试结果与总结

5.2.1 测试内容及结果

该系统测试部分主要验证 P2P 网络是否通畅、用户的注册和登录是否成功、签名和验证是否成功、数据同步是否及时和准确、文本信息的加解密是否成功、DPoS 是否运行正常等。其余大部分功能，该系统已经实现了自我验证。

在运行测试前，需要先配置多个应用节点，模拟出该系统的 P2P 网络环境。最简单的方法是在不同窗口运行程序，配置不同端口和数据库文件即可。当然还可以通过运行多个 Docker 容器来模拟，或者在多台电脑上还原真实的使用场景进行试验。以下是该系统部分测试的内容、方法及结果。

(1) 密钥协商测试

该功能的测试主要验证文本信息的发送双方，是否能协商出完全一致的共享密钥，以保证接收方能解开发送方发出的密文。测试结果如图 5-3 所示，从图中可以看出，用 A 的私钥与 B 的公钥协商出的共享密钥 shareKey1，和用 B 的私钥与 A 的公钥协商出的共享密钥 shareKey2，结果完全一致。

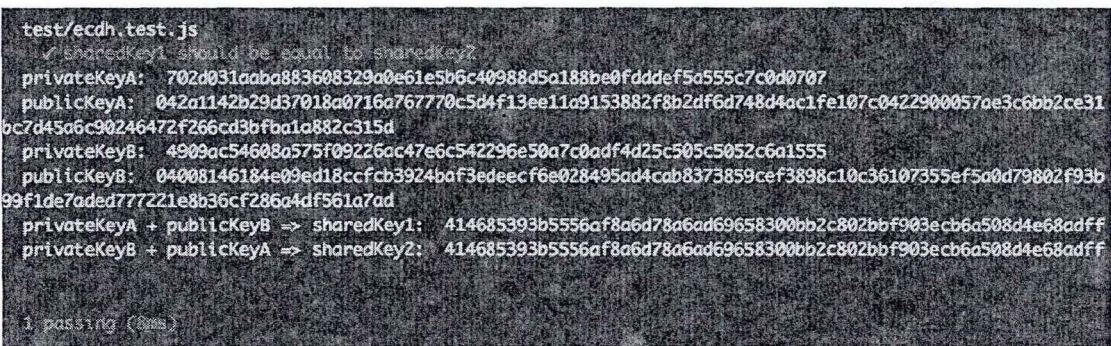


图 5-3 密钥协商测试结果截图

(2) 网络测试

该部分测试主要验证区块链网络中的节点是否初始化成功，节点信息是否同步，

能否返回正确的节点信息。具体的，分别请求某一节点看是否返回当前正在运行的节点列表信息，并将该列表信息暂存起来作为参考，然后根据返回的节点列表，逐一验证其他节点，并验证各返回的列表信息是否与参考列表一致。

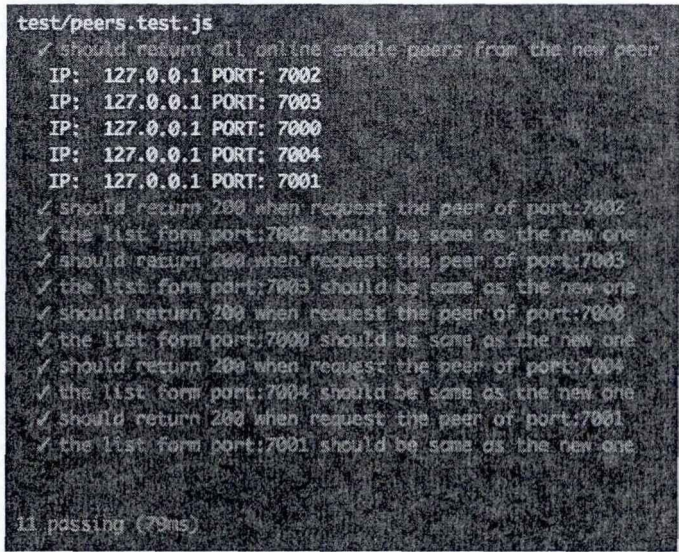


图 5-4 网络测试结果截图

测试结果如图 5-4 所示。从结果可以看出，不同节点能够正确返回节点列表，节点均能够请求成功且返回的列表数据完全一致，证明该 P2P 网络通畅，通信正常。

(3) 用户注册及登录

该部分测试主要模拟用户注册，验证私钥的加密备份是否成功，验证能否通过用户名获取用户信息，验证能否解密出私钥，验证解密出的私钥是否正确。

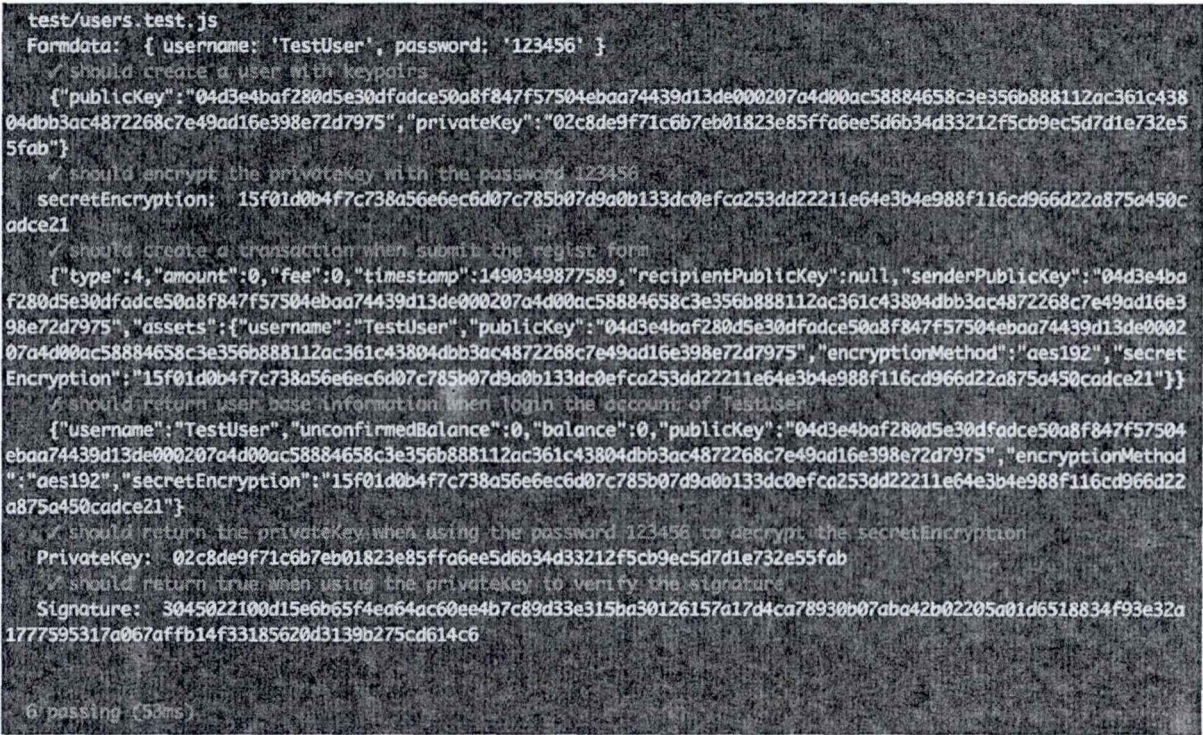


图 5-5 用户注册和登录测试结果截图

测试结果如图 5-5 所示，从测试结果可以看出，用户注册和登录功能正常。同时 aes192 对称加密和签名验证功能均测试通过，说明该系统的对称加密和签名验证均能正常工作，在交易、区块、文本信息等处理上同样适用。

(4) 区块数据同步测试

该部分测试主要验证区块数据是否同步成功。测试前，新创建一个初始节点，以验证区块数据更新相对比较落后的情况。测试时，首先加载新创建节点的区块数据。然后分别请求获取每个节点的区块数据，并从创世区块开始，分别验证各个节点同一块高度的区块数据是否相同，包括 ID 是否相同、时间戳是否一致，以及区块中的交易数据是否完全一致等。当然，最好能在有新的区块创建时，自动完成该测试。测试结果如图 5-6 所示。

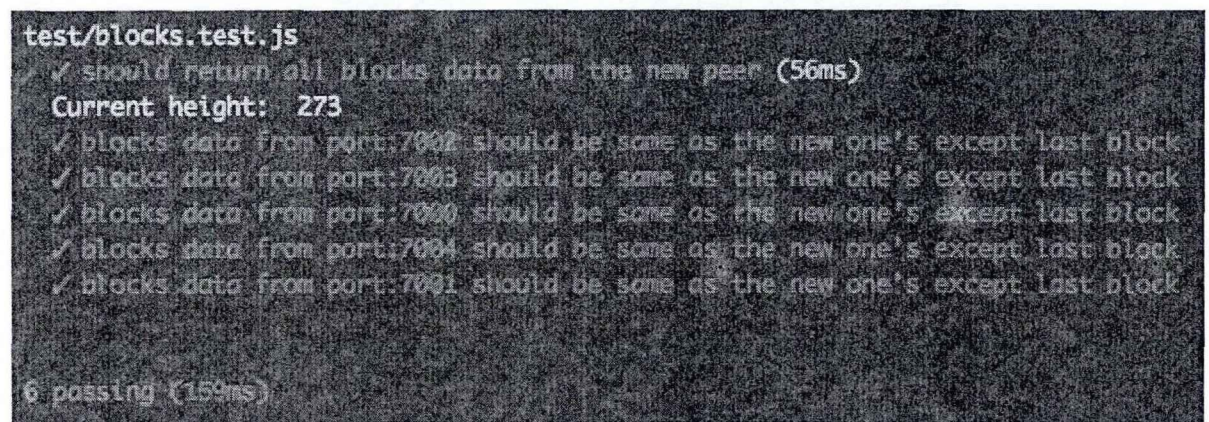


图 5-6 区块数据同步测试结果截图

从测试结果可以看出，包括最新创建的节点在内的所有节点的区块数据完全一致，数据能够实时同步，每个节点都具有区块数据的完整备份。

(5) DPoS 测试

DPoS 机制贯穿整个系统始终，它是该系统核心控制器。由于该模块涉及的业务多，关联性强，并且大多数业务在系统各个环节做了自我验证，所以该部分的测试主要通过人工验证来进一步证实系统产生的结果正确。测试代码主要是对部分网络接口进行正确性验证，此处不再赘述。如图 5-7 是 DPoS 的日志的截图，通过日志可以看出整个应用在该机制的管理下正常进行。系统每隔十分钟产生一个区块，区块数据基本正确，满足设计的预期要求。

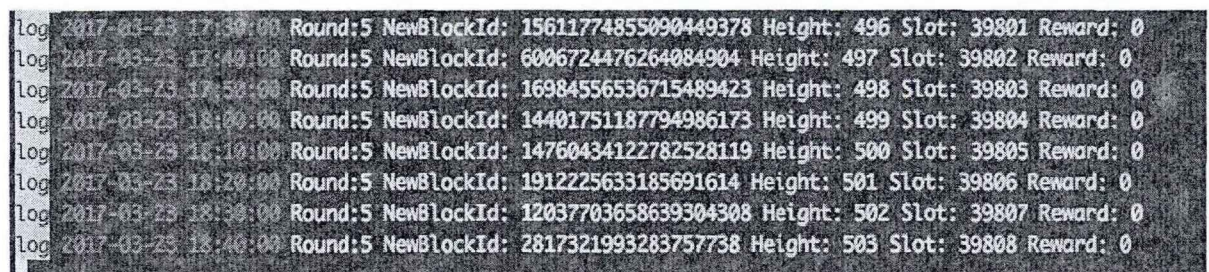


图 5-7 DPoS 日志截图

5.2.2 测试结果总结

综合上述测试内容及结果，可见该加密备份系统已经满足基本功能需求，各个模块的主要方法能正常工作并返回正确的结果。该系统没有对性能进行测试，有待进一步完善，将根据性能测试数据，分析和梳理业务代码，利用更高效的实现方式，降低性能消耗，提高系统响应速度。

5.3 本章小结

本章主要介绍了该文本加密备份系统的测试模块，首先介绍了测试的基本概念，然后介绍了测试所用到的测试工具，以及如何利用这些工具搭建测试环境并编写测试代码，最后介绍了测试流程、测试内容及结果，并对结果进行了简单总结。该系统还有进一步优化的空间，持续写测试有助于编写稳定高效执行的代码，避免新的修改导致旧代码的功能错误，是软件开发过程中非常有价值的时间投入。

第6章 总结与展望

本文是对区块链技术的研究与应用创新，具有重要的研究价值和应用价值。本文设计了基于区块链的加密信息备份系统，并采用 Node.js 技术完成了该系统的开发和测试，主要包括 P2P 网络构建、签名和验证、账户模块、交易数据产生和广播、区块数据同步和验证、共识机制等基础功能，以及用户私钥加密备份、文本信息加密备份和信息安全发送等扩展功能。本文通过设计基于 P2P 网络的分布式数据存储策略，保证了用户备份的文本信息不会丢失；采用基于椭圆曲线系列算法的签名验证技术，实现了信息的所有权认证；采用安全的加解密技术，保证了用户信息的安全；设计了有效的共识机制，保证系统正常运行；设计了可远程访问的应用节点，使得用户可以随时随地操作文本信息。该系统如果在一定数量的用户中使用起来，将构建一个非常安全可靠的区块链网络，帮助用户在全球范围内分布式备份加密的文本信息。

本文在设计上还存在以下几个方面不足：

(1)没有合理地分割区块数据，一旦数据急速膨胀，会造成存储资源的浪费，数据同步成本也大大增加；

(2)共识机制的设计还很不完善，需要在实际使用过程中，总结其他区块链应用的经验，对共识策略进行改进；

(3)文本信息的备份和发送虽然解决了安全可信的问题，但需要等待交易确认，失去了信息传播的实时性。

面对这些不足，需要大胆创新，敢于尝试。比如，为了解决区块数据分割的问题，需要设计某种机制，以区块为最小数据单位，将交易数据分散存储在全网节点中。根据某种映射算法，指定要求哪些节点必须存储某个区块的备份，或者哪些旧区块数据可以被某节点清除。每个节点都保留有最新区块信息，这样就可以不断通过网络往前追溯找出目标区块，并从目标节点提取交易信息。这在理论上解决了数据膨胀的问题，而实际使用时可能会出现频繁的网络请求，造成网络拥堵的新问题，可以通过缓存技术、网络技术、改进数据结构设计等途径进一步优化。

本文只是对区块链研究的一小步，在后续的研究中，还需要思考和改进系统设计中的不足，逐步完善系统的各个细节。通过总结用户反馈意见，提出技术改进方案，根据测试数据进行综合评估，提出性能优化措施。把每一种方案落实到实践，以强有力的实验数据为支撑，选择更好更实际的解决方法。

本文从该系统的设计和开发上，验证了区块链在去中心化信息管理方面的技术可行性和安全可靠。区块链的诞生和发展，有其存在的必然意义，它从技术上解决了去中心化、全球互信等核心问题。以区块链技术为代表的去中心化技术思想，定将引领一个全新的信息时代。

致 谢

时光荏苒，岁月如梭。三年的研究生学习生涯伴随毕业论文的收尾，也将画上圆满的句号。三年时间，在任课老师的教导下，在实验室老师的指引下，我深深的体会到学术研究的魅力和重要性，从中懂得了诚实做人、踏实做事的道理。三年时间，在实验室工作中和实习单位的历练中，我将基础知识与项目实践相结合，使自己的知识面得到了非常丰富的拓展，技术水平也有明显的提升，面对今后的就业没有任何压力。三年时间，我与学院的老师、同学友好相处，互相交流学习，一起旅游探险，茶余饭后间建立了非常难忘的革命友谊。匆匆三年，有太多要感恩的人和事，伴我成长，伴我成熟。

首先，感谢我的导师李建老师，从论文的选题到最后的定稿，每个阶段都得到了李老师的细心指导，帮助改进论文中的不足之处。研究生三年，李老师教会我许多为人处世的道理，给予我充分的理解、包容、支持以及无微不至的关心和问候，让我特别感动。在此向李老师表示衷心的感谢和诚挚的敬意。

同时，感谢实验室的王兵老师和肖斌老师，以及给我授过课的任课老师们，你们的为人师表，治学严谨，平易近人，充分感染了我，让我感受到浓浓的师生情谊；感谢一直关心并支持我的同学和朋友，在我困难的时候，在我无助的时候，在我孤独的时候，给了我贴心的陪伴和及时的帮助；感谢我的母校，赐予我这么好的学习环境和教学资源，赐予我这么好的老师和同学，感恩这里的一草一木，以及在这里求学的每一寸光阴。

再次感谢关心、帮助、支持我的所有人，祝你们工作顺利，身体健康！

参考文献

- [1] 张健. 区块链:定义未来金融与经济新格局[M]. 北京:机械工业出版社,2016:30-229.
- [2] 徐明星,刘勇,段新星,等. 区块链:重塑经济与世界[M]. 北京:中信出版社,2016:4-217.
- [3] Melanie Swan. Blockchain:Blueprint for a New Economy. 2015.
- [4] 袁勇,王飞跃. 区块链技术发展现状与展望[J]. 自动化学报,2016,(04):481-494.
- [5] 蔡维德,郁莲,王荣,等. 基于区块链的应用系统开发方法研究[J]. 软件学报,2017.
- [6] 朱岩,甘国华,邓迪,等. 区块链关键技术中的安全性研究[J]. 信息安全研究,2016,(12):1090-1097.
- [7] 石菲. 区块链:打造诚信社会[J]. 中国信息化,2016,(12):50-51.
- [8] 蒋润祥,魏长江. 区块链的应用进展与价值探讨[J]. 甘肃金融,2016,(02):19-21.
- [9] 沈鑫,裴庆祺,刘雪峰. 区块链技术综述[J]. 网络与信息安全学报,2016,(11):11-20.
- [10]王硕. 区块链技术在金融领域的研究现状及创新趋势分析[J]. 上海金融,2016,(02):26-29.
- [11]李政葳. 区块链技术将如何改变生活[N]. 光明日报,2016-02-16(006).
- [12]胥月,马小峰. 基于区块链的学生行为综合评价体系的研究与实现[J]. 信息技术与信息化,2016,(12):131-133.
- [13]张楠. 区块链技术在金融创新中的应用[N]. 金融时报,2017-02-13(011).
- [14]骆慧勇. 区块链技术原理与应用价值[J]. 金融纵横,2016,(07):33-37+76.
- [15]谭磊,陈刚. 区块链 2.0[J]. 中国信息化,2016,(08):97.
- [16]Satoshi Nakamoto. Bitcoin:A Peer-to-Peer Electronic Cash System [Online]. <https://www.bitcoin.org/bitcoin.pdf>,2008.
- [17]Bitcoin Sourcecode[Online]. <https://github.com/bitcoin/bitcoin/>. 2016.
- [18]中国区块链技术和应用发展白皮书[Online]. <http://8btc.com/doc-view-984.html>. 2016.
- [19]王华.《贵阳区块链发展和应用》白皮书正式发布[N]. 贵阳日报,2017-01-01(002).
- [20]Claude E.Shannon. Communication Theory of Secrecy Systems[J]. Bell System Technical Journal,1949,(04):656-715.
- [21]Whitfield Diffie, Martin E.Hellman. New Directions in Cryptography[J]. IEEE

Transactions on Information Theory, 1976,(06):644-654.

[22]张秀娟. 基于 AES 加密算法的实现与应用[J]. 数字技术与应用,2011,(12):128+131.

[23]张金辉,郭晓彪,符鑫. AES 加密算法分析及其在信息安全中的应用[J]. 信息网络安全,2011,(05):31-33.

[24]王艳红. 基于椭圆曲线密码数字签名的研究与实现[J]. 信息通信,2011,(03):17+58.

[25]任艳芳. 基于椭圆曲线密码(ECC)的数字签名技术[J]. 硅谷,2013,(12):51-53.

[26]任中岗. 椭圆曲线算法研究与应用[D].西南交通大学,2009.

[27]朱利民,王明斐. 基于椭圆曲线算法的数字签名技术研究[J]. 甘肃科技,2010,(04):40-41+3.

[28]张岩. 基于椭圆曲线的数字签名算法研究[D].西南交通大学,2010.

[29]于伟. 椭圆曲线密码学若干算法研究[D].中国科学技术大学,2013.

[30]葛宏华,丁秀欢. 基于 AES 和 ECC 的混合密码体制[J]. 科技信息,2011,(09):455-457.

[31]繆昌照,徐俊武. AES 与 ECC 混合加密算法研究[J]. 软件导刊,2016,(11):63-64.

[32]周杰. 基于 AES 和 ECC 的加密体制研究及硬件实现[D]. 西安电子科技大学,2011.

[33]吴晶晶. PKI 关键理论与应用技术研究[D]. 中国科学技术大学,2008.

[34]胡云. RSA 算法研究与实现[D]. 北京邮电大学,2010.

[35]崔苗,姚震. 基于 ELGamal 公钥密码的算法分析与实现[J]. 福建电脑,2006,(04):120-121.

[36]方妮. DSA 数字签名算法研究及软件实现[A]. 中国工程物理研究院科技年报,2001.

[37]田松,李宝,王鲲鹏. 椭圆曲线离散对数问题的研究进展[J]. 密码学报,2015,(02):177-188.

[38]张伟. ECDSA 算法实现及其安全性分析[J]. 信息与电子工程,2003,(02):7-12.

[39]范云海. 集成加密方案 ECIES 的设计与验证[J]. 信息技术,2012,(01):115-117.

[40]李胜金,张昌宏,周大伟. 一种基于 ECDH 的可认证密钥协商协议[J]. 信息安全与通信保密,2011,(07):70-72.

- [41]蒋林涛. P2P 技术的分析与研究[J]. 电信网技术,2007(3).
- [42]郭玉堂. 基于 P2P 技术的对等网络关键技术研究与分析[J]. 安徽教育学院学报,2005,(06):43-46.
- [43]陈姝,方滨兴,周勇林. P2P 技术的研究与应用[J]. 计算机工程与应用,2002(13).
- [44]张一兰. 去中心化的力量——互联网时代下组织架构及管控模式重塑[J]. 人力资源管理,2015,(02):22.
- [45]李哲宇. 比特币——去中心化货币的可能未来[J]. 中国集体经济,2017,(03):99-100.
- [46]赵亮. 高性能磁盘阵列 (RAID) 关键技术的研究[D]. 国防科学技术大学,2002.
- [47]Daniel Larimer. Delegated proof-of-stake white paper[Online]. <http://www.bts.hk/dpos-baipishu.html>. 2014.
- [48]王金龙,宋斌,丁锐. Node.js:一种新的 Web 应用构建技术[J]. 现代电子技术,2015,(06):70-73.
- [49]陆凌牛. Node.js 权威指南[M]. 北京:机械工业出版社,2014:18-499.
- [50]刘生建,罗林,杜剑. Node.js 高效分布式实时应用研究[J]. 软件导刊,2014,(12):25-28.
- [51]李林,杨先文. Cryptlib 密码服务库改进与实现[J]. 计算机应用研究,2012,(12):4662-4665.
- [52]Crypto [Online]. <https://nodejs.org/api/crypto.html/>.
- [53]eccrypto[Online]. <https://www.npmjs.com/package/eccrypto/>.
- [54]宋洁. 软件测试技术及自动测试工具的实现[D]. 北京邮电大学,2007.
- [55]呼晓黎. 软件自动化测试技术的研究与实现[D]. 西南交通大学,2007.
- [56]蔡高亮. 软件单元测试[J]. 信息技术与标准化,2008,(Z1):41-43.
- [57]余艳. 软件单元测试技术研究[D]. 华中科技大学,2004.